



XbMonitor

©2008 caryHumphries

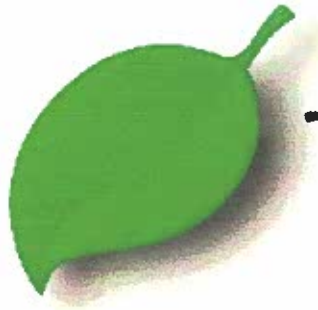


Table of Contents

Introduction

- Project Information
- Purpose and Background
- System Requirements

System Analysis & Design

- Problem Statement
- Project Proposal
- Goals and Objectives
- System Diagram
- Problems and Constraints

System Functions & Algorithms

- System Device Discovery Algorithm
- Threaded Data Update Functions

System Screens & Code

- Main Status Window
- Inverter/Charger
- Solar Charge Controller
- Battery Storage Module
- Automated Generator Start Panel
- System Control Panel

Project By:

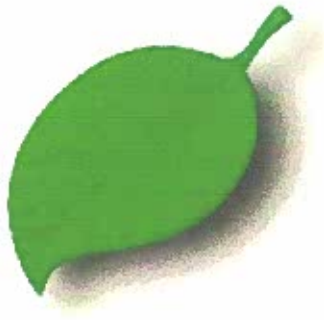
Cary M. Humphries
1329 Pinyon Pine Drive
Ladson, SC 29456
843.574.9933
cary_humphries@hotmail.com

Project Advisor:

Dr. John Sullivan
Professor of Computer Science
Charleston Southern University
jsullivan@csuniv.edu

Project Contributors:

Xantrex Technology Inc.
<http://www.xantrex.com/xw/>
Apox Controls, LLC
<http://www.apoxcontrols.com>



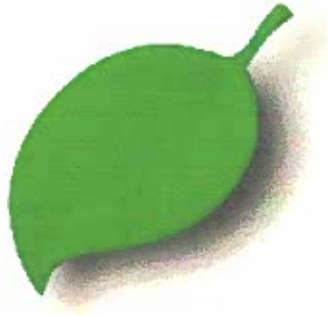
Purpose and Background

Purpose

The XbMonitor program is a software-based system created for the Xantrex™ XW series of products. The Xantrex™ XW products are used to control renewable energy and backup power systems. These Xantrex™ XW products communicate with each other through a built-in controller area network called Xanbus™. The XbMonitor program is designed to connect directly to the Xanbus™ network by using a simple USB connection and an Apox™ USB to CAN converter. When connected to the network the program is able to discover, monitor, and communicate with all attached devices from a single connection point while displaying almost real-time data in a user friendly graphical display.

Background

The Xantrex™ XW series of products were designed with the ability to communicate directly with each other as a complete integrated system. Unfortunately, the only way to monitor the devices on the Xanbus™ network is through the proprietary, hardware-based System Control Panel. The System Control Panel is a great way to configuring a system, but proves cumbersome when it comes to monitoring. While researching the Xantrex™ products because of my own interest in renewable energy, I have found many owners requesting some other way to monitor their systems. This is where the idea for the XbMonitor program originated.



System Requirements

Operating Systems Supported

Windows 98 or later

Software Required

Python 2.4

- <http://www.python.org/download/>

FTDI USB-CAN Driver v2.00

- <http://www.ftdichip.com/Drivers/D2XX.htm>

Python XanBus API ~ ©Xantrex Technology Inc.

- PyXanBus-1.0build726.win32-py2.4.exe

Hardware Required

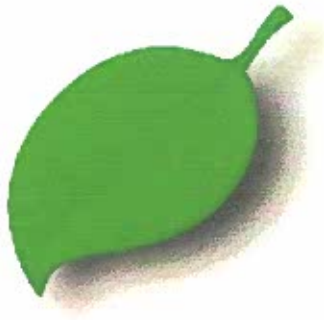
Apox Controls USB to CAN converter

- <http://www.apoxcontrols.com/CARTgallery.htm>

Xanbus connection (9-Pin D-sub to RJ-45 Male)



System Analysis & Design



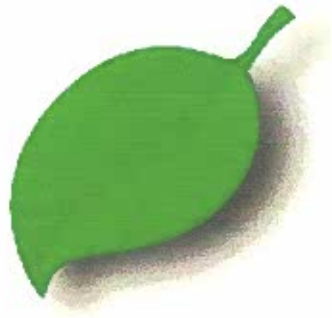
System Analysis & Design

Problem

Current monitoring systems for renewable energy and backup power systems either do not provide an easy way to communicate with multiple devices or they provide a limited view of the system status. The Xantrex™ XW system is the only available product line that incorporates a device-to-device controller area network built into each device. However, there is no easy way to communicate with or monitor the devices on a Xanbus™ network.

Because of the difficulties involved to establish a connection to the Xanbus™ network, the average Xantrex™ XW system owner is not able to effectively monitor their own equipment. However, Xantrex™ does offer free software to connect to individual system devices using a personal computer's serial port and an RS-232 cable, but this only allows communication with one device at a time and provides limited information.

A user-friendly system monitoring program does not exist at this time from Xantrex™ or any other source. There is a demand for a computer based Xanbus™ network monitoring system based on customer requests made on the Xantrex™ customer support and developer message boards.



System Analysis & Design

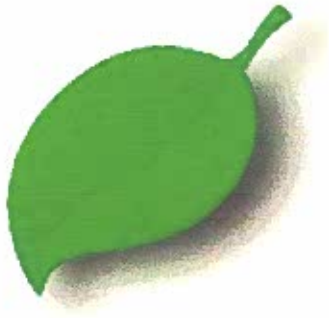
Proposal

Development of a software-based monitoring system that would connect to a Xanbus™ network using a USB to CAN adapter would allow the system to monitor and communicate with all of the devices on the network simultaneously. With access to the network, system and device information could be collected and displayed in real-time with an easy to use graphical interface.

By using an application interface to the Xanbus™ network a graphical user interface could be designed that communicates directly with the Xantrex™ XW devices. Once an initial monitoring system is established, future extended capabilities are possible such as historical data logging, usage graphs, and webpage interfaces.

Plan

1. Research the Xanbus™ API and CANbus implementation
2. Establish Xanbus communication protocols for functions
3. Define functions necessary to implement the system
4. Design the user interface displays
5. Implement data algorithms for the user interface
6. Enhance algorithms using threads to collect data
7. Complete documentation and finalize project



System Analysis & Design

Goals & Objectives

The overall goal of this project is to design, test, and implement a functional software monitoring program for a Xantrex™ XW power management system using the Xanbus™ network.

Software design will be accomplished using Python in order to be compatible with the application interface provided by Xantrex™. The user interface will be written using Python's tk interface.

Project Duration:

32 weeks (Sept. 2007 – April 2008)

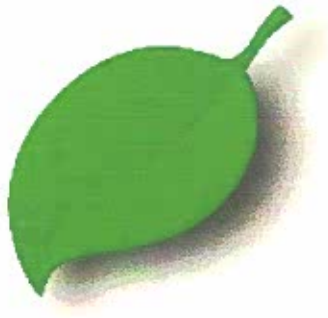
Milestones:

Weeks 1–8: Research, Analysis & Design

Weeks 9–17: Construction (Coding functions and GUI)

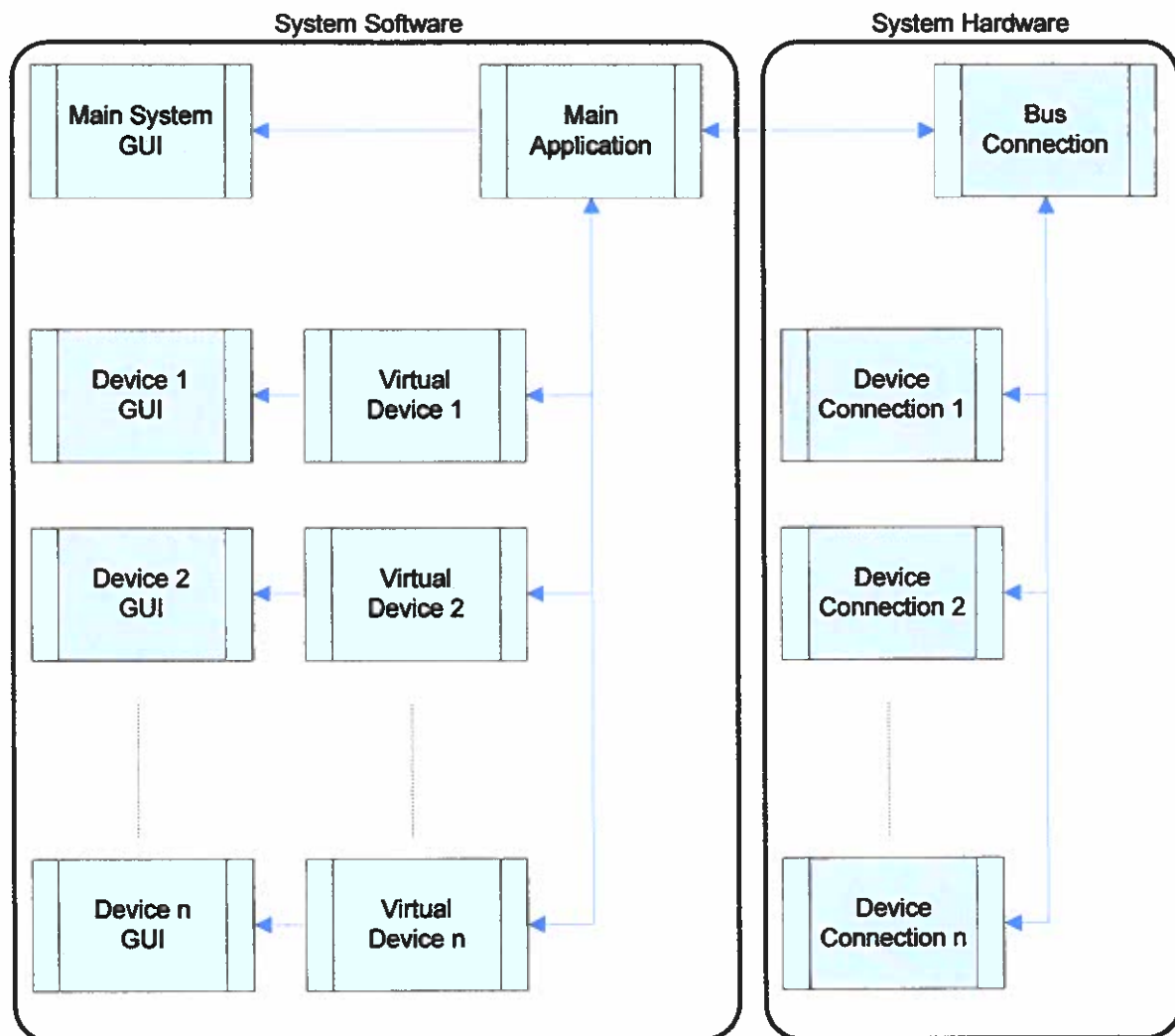
Weeks 18–27: Implementation (Testing & Enhancing code)

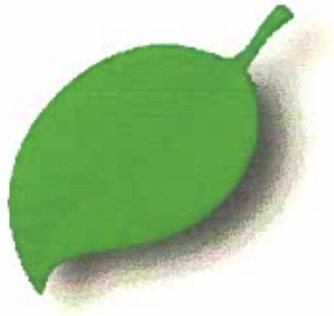
Weeks 28–32: Documentation and Presentation



System Analysis & Design

System Diagram



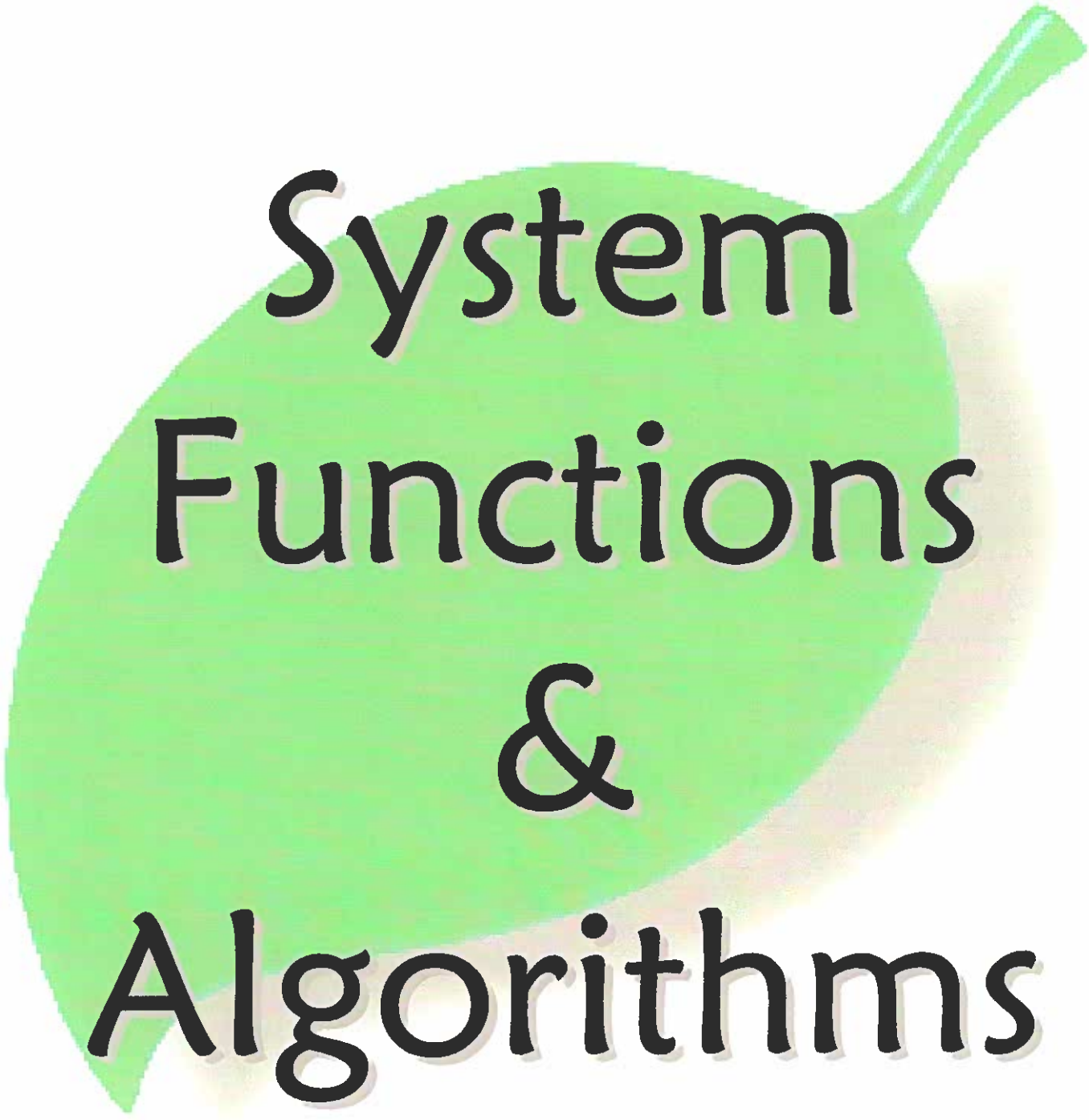


System Analysis & Design

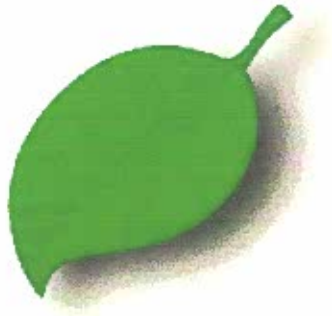
Problems & Constraints

Problem	Effect
1. There are currently no algorithms to perform device discovery on the Xanbus™ network.	1. Logic and algorithms built from scratch. Without the functions the program would not function.
2. The Xanbus API does not support any graphical interfaces.	2. Python tkinterface graphic interfaces had to be created by hand.
3. Minimal Xanbus API documentation exists.	3. Time consuming trial and error testing was required to complete algorithms.

Objective	Constraint
1. Device data logging.	1. Future implementation, time not available.
2. Usage history charts.	2. No data logging function.



System Functions & Algorithms

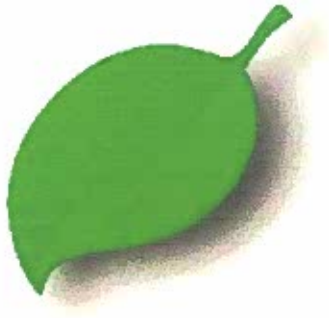


System Functions & Algorithms

Device Discovery

The device discovery algorithm required careful planning because each device required a distinct GUI class to be used depending on what types and numbers of devices were found on the network. A unique device GUI window is created for each device discovered and the device is added to the devCfg list for later menu creation for access to the device GUI windows.

```
if devices:
    devCount = 0
    for address, claim in devices:
        devCon = PyXanBus.clDeviceConnection(busCon, address, 0)
        message = devCon.fnRequest(0x13A00)
        devFunc = message[0].fnGetValue('DeviceFunction')
        serNum = message[0].fnGetValue('UniqueNumber')
        if devFunc == 'User Interface':
            devCfg.append((address, devFunc, serNum, ScPnl(Toplevel())))
            devCon.fnClose()
        elif devFunc == 'Inverter Charger':
            devCfg.append((address, devFunc, serNum, InvChg(Toplevel())))
            # Find out if this is the master inverter for the system
            # If it is then set the masterAddr and create a battery window
            message = devCon.fnRequest(0x1F0BD)
            mode = message[0].fnGetRawValue('InvMode')
            MasterModes = [1, 11, 20, 21, 23, 25, 30, 31, 33, 35, 37]
            if mode in MasterModes:
                devCfg.append((address, 'Battery', 0000000, Battery(Toplevel())))
                XBMonitor.setAddress(address)
            else: pass
            devCon.fnClose()
        elif devFunc == 'Charger':
            devCfg.append((address, devFunc, serNum, SolChg(Toplevel())))
            devCon.fnClose()
        elif devFunc == 'Battery Monitor':
            devCfg.append((address, devFunc, serNum, Battery(Toplevel())))
            devCon.fnClose()
        elif devFunc == 'AGS':
            devCfg.append((address, devFunc, serNum, GenPnl(Toplevel())))
            devCon.fnClose()
        else: pass
```



System Functions & Algorithms

Threaded Data Update Functions

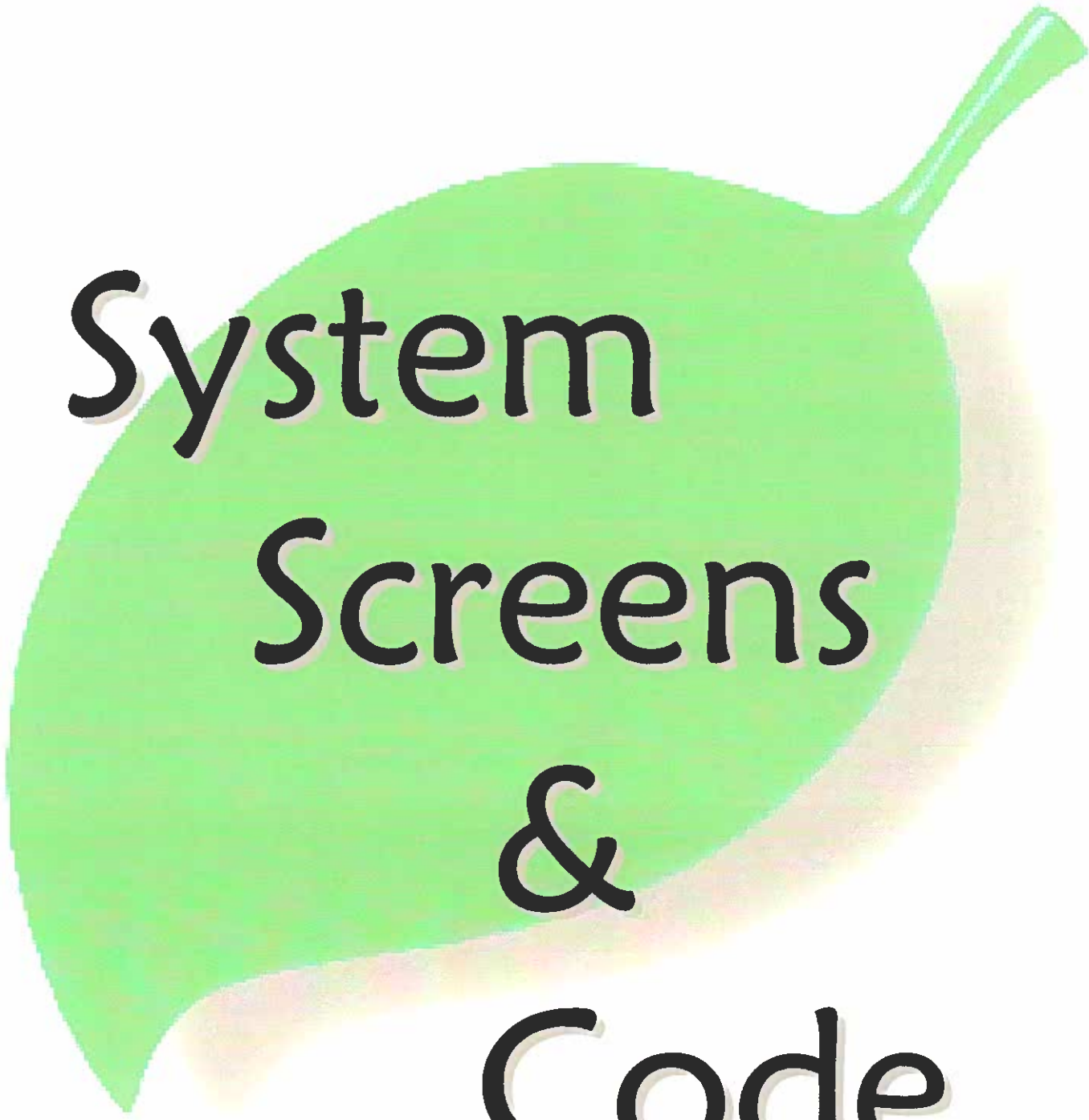
The devices found in the devCfg list then get their GUI windows customized and event handlers are bound to them to prevent accidental destruction of the data and window. The escKeyHandler function prevents the window from being destroyed and re-routes the command to hide the device GUI window instead.

Once the GUI windows are finished, the network address for the device and its serial number are sent to the virtual devices setNameCfg function.

Finally, the virtual device worker function is started on a thread to update the data in the background at 5 second intervals.

Each virtual device has a unique worker thread function that uses the actual device network address to retrieve the data specific to that device type.

```
if devCfg:
    for i in range(0, len(devCfg)):
        devCfg[i][3].winfo_toplevel().title(devCfg[i][1] + ' (esc to close)')
        devCfg[i][3].winfo_toplevel().iconbitmap('Coda.ico')
        devCfg[i][3].winfo_toplevel().withdraw()
        def escKeyHandler(event, self=devCfg[i][3]):
            self.noEscape()
        devCfg[i][3].winfo_toplevel().bind('<Key-Escape>', escKeyHandler)
        devCfg[i][3].winfo_toplevel().protocol('WM_DELETE_WINDOW', lambda:0)
        devCfg[i][3].winfo_toplevel().resizable(width=False, height=False)
        deviceMenu.add_command(label=devCfg[i][1] + ':%s' %devCfg[i][0], \
                               command=devCfg[i][3].winfo_toplevel().deiconify)
        devCfg[i][3].setNameCfg(devCfg[i][2], devCfg[i][0])
        thread.start_new(devCfg[i][3].worker, ())
else: pass
```



System Screens & Code



The screenshot shows a 'System Monitor' window with a blue title bar and standard Windows window controls. The main content area has a yellow header with 'Commands' and 'Device:Address'. Below this, there are four sections: 'AC Input', 'Inverter', 'AC Output', and 'Battery'. Each section displays various status values in a table-like format. The 'AC Input' section shows 'Invalid' for both 'Device' and 'Address' columns, with values of 0.0 for Volts, Amps, Watts, and Hertz. The 'Inverter' section shows 'State' as 'Load Sense' and 'Mode' as 'Invalid'. The 'AC Output' section shows 'AC Load 1' with values of 0.0 for Volts, Amps, Watts, and Hertz. The 'Battery' section shows values of 0.0 for Volts, Amps, Watts, Temp (F), Capacity, and Wh Remaining. At the bottom, a yellow bar displays 'Bus API Version: 1.4.0 build 726'.

Commands		Device:Address	
AC Input			
Invalid		Invalid	
0.0	Volts	0.0	
0.0	Amps	0.0	
0.0	Watts	0.0	
0.0	Hertz	0.0	
Inverter			
State		AC Load 1	
Load Sense		0.0	
Mode		0.0	
Invalid		0.0	
AC Output			
Volts		0.0	
Amps		0.0	
Watts		0.0	
Hertz		0.0	
Battery			
Volts	Amps	Watts	Temp (F)
0.0	0.0	0.0	0.0
Capacity		Wh Remaining	
0.0		0.0	
Bus API Version: 1.4.0 build 726			



System Screens & Code

Inverter / Charger Window

Inverter Charger (esc to close)

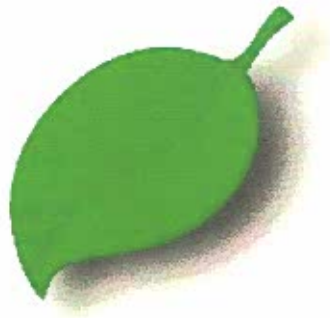
Inverter/Charger

XW Inverter Charger XW6048-120/240-60
Unique ID 12345678

Inverter		Charger	
AC Load	Invalid	DC Source	Invalid
State	Engaging Inverter	State	Not Charging
Mode	Invalid	Mode	Standalone
Enabled	On	Enabled	On
Grid Enabled	On	Algorithm	Invalid
Sell Enabled	On	Control Voltage	0.0
Force Sell	Off	Control Amps	0.0
Energy Today (Wh)	0.0	Maximum Amps(%)	0.0

AC Line 1		AC Line 2	
AC Source 1	Line 1	AC Source 2	Line 2
Source Id	Invalid	Source Id	Invalid
Quality	Invalid	Quality	Invalid
AC Volts	0.0	AC Volts	0.0
AC Amps	0.0	AC Amps	0.0
AC Freq(Hz)	0.0	AC Freq(Hz)	0.0
Apparent Power(VA)	0.0	Apparent Power(VA)	0.0
Real Power(W)	0.0	Real Power(W)	0.0

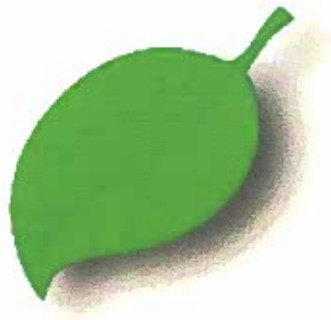
AC Load		DC Source	
AC Output	Invalid	DC Source	Invalid
AC Output Volts	0.0	DC Volts +/-	0.0
AC Output Amps	0.0	DC Amps +/-	0.0
AC Maximum Amps(%)	0.0	DC Watts +/-	0.0
AC Output Freq(Hz)	0.0	(Negative value indicates charging)	
AC Output Waveform	Invalid		
Apparent Power(VA)	0.0		
Real Power(W)	0.0		
Real Power(%)	0.0		



System Screens & Code

Solar Charge Controller Window

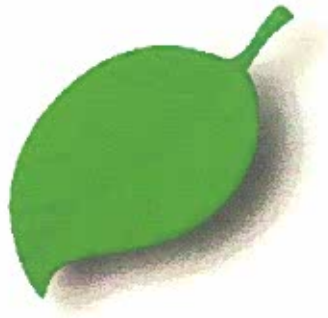
Charger (esc to close)			
Solar Charge Controller			
MPPT-60		XW-MPPT60-150	
ID #	23456789	Control Voltage	0.0
DC Source	Invalid	Control Amps	0.0
Current State	Not Charging	Control Amps %	0.0
Current Mode	Standalone	DC Volts +/-	0.0
Enabled	On	DC Amps +/-	0.0
Algorithm	Invalid	Watts +/-	0.0



System Screens & Code

Battery Storage Module Window

Battery (esc to close)			
Battery Storage			
Invalid			
DC Volts +/-	0.0	State of Charge	0.0
DC Amps +/-	0.0	State of Health	0.0
DC Watts +/-	0.0	Time Remaining	0.0
Temp C/F	0.0	WattHours Remaining	0.0
Capacity Ah	0.0	Time To Full	0.0
Capacity %	0.0	Time Since Full	0.0



System Screens & Code

Automated Generator Start Window

 AGS (esc to close)   

Generator Control Panel

XW-AGS		XW Automatic Generator Start	
		ID #	34567890
Source	Invalid	Control Signal	Off
Generator Mode	Not Operating	Generator State	Stopped
Reason On	Not On	Reason Off	Auto Off

System Control Panel Window

 User Interface (esc to close)   

System Control Panel

Model Name	SCP
ID #	98765432

```
#####
#
# File: XbMonitor.py(w)      Description: A Monitoring utility to display Xanbus device status
# Date: April 2008
# Author: Cary Humphries    Assumptions: This program was written for an Apox USBCAN dongle
# email: cary_humphries@hotmail.com    connected to a Xanbus network.
#
#####

from Tkinter import *
import PyXanBus, thread, time, tkFont
from ScrolledText import ScrolledText
from Devices import AcInpt, \
    AcOutput, \
    BattState, \
    InvState, \
    InvChg, \
    Battery, \
    SolChg, \
    GenPnl, \
    ScPnl

# List for the initial device windows and menu setup
devCfg = []
# Device bus connection object to the CanBus network
busCon = PyXanBus.clBusConnection()
# List of devices on the CanBus network
devices = busCon.fnDevices()

# Class for the main status window of the XBMonitor application
class App(Frame):
    def __init__(self, master=None):
        Frame.__init__(self, master)
        self.grid()
        self.masterAddr = IntVar()
        self.makeDisplay()

    def makeDisplay(self):
        self.masterAddr.set(0)
        self.AcIn = AcInpt(self)
        self.Inverter = InvState(self)
        self.AcOut = AcOutput(self)
        self.Batt = BattState(self)
        self.AcIn.grid(row=1, column=0)
        self.Inverter.grid(row=1, column=1)
        self.AcOut.grid(row=1, column=2)
        self.Batt.grid(row=2, column=0, columnspan=3)

    def getAddress(self):
        return self.masterAddr.get()

# Sets the masterAddress variables to the address of the master inverter
def setAddress(self, addr):
    self.masterAddr.set(addr)
    self.AcIn.setAddr(addr)
    self.Inverter.setAddr(addr)
    self.AcOut.setAddr(addr)
    self.Batt.setAddr(addr)

# Exit function for the application
def close(self):
    busCon.fnClose()
    self.quit()

```

```

# Main Status Window Creation
XBMonitor = App()
XBMonitor.master.title('System Monitor')
XBMonitor.master.iconbitmap('Coda.ico')
XBMonitor.master.resizable(width=False, height=False)

# Main status window destructor override
# You must use 'Quit' from the file menu to prevent hanging the bus connection
XBMonitor.master.protocol('WM_DELETE_WINDOW', lambda:0)

# Menus for the Main Status Window
menubar = Menu(XBMonitor.master)
commandMenu = Menu(menubar,tearoff=0)
deviceMenu = Menu(menubar,tearoff=0)
menubar.add_cascade(label='Commands',menu=commandMenu)
menubar.add_cascade(label='Device:Address',menu=deviceMenu)
commandMenu.add_command(label='Quit!',command=XBMonitor.close)
#####
# BEGIN SYSTEM DEVICE DISCOVERY
# Dynamically create the virtual device monitors and windows for each device
# found in the CanBus device list. Each device is queried to determine
# what type of device it is, and then an appropriate object is created to
# store and display that devices information. The bus address, device
# function, serial number, and object memory address for each device is then
# stored as a tuple in the devCfg list.
#
#####
if devices:
    devCount = 0
    for address, claim in devices:
        devCon = PyXanBus.clDeviceConnection(busCon, address, 0)
        message = devCon.fnRequest(0x13A00)
        devFunc = message[0].fnGetValue('DeviceFunction')
        serNum = message[0].fnGetValue('UniqueNumber')
        if devFunc == 'User Interface':
            devCfg.append((address, devFunc, serNum, ScPnl(Toplevel())))
            devCon.fnClose()
        elif devFunc == 'Inverter Charger':
            devCfg.append((address, devFunc, serNum, InvChg(Toplevel())))
            # Find out if this is the master inverter for the system
            # If it is then set the masterAddr and create a battery window
            message = devCon.fnRequest(0x1F0BD)
            mode = message[0].fnGetRawValue('InvMode')
            MasterModes = [1, 11, 20, 21, 23, 25, 30, 31, 33, 35, 37]
            if mode in MasterModes:
                devCfg.append((address, 'Battery', 0000000, Battery(Toplevel())))
                XBMonitor.setAddress(address)
            else: pass
            devCon.fnClose()
        elif devFunc == 'Charger':
            devCfg.append((address, devFunc, serNum, SolChg(Toplevel())))
            devCon.fnClose()
        elif devFunc == 'Battery Monitor':
            devCfg.append((address, devFunc, serNum, Battery(Toplevel())))
            devCon.fnClose()
        elif devFunc == 'AGS':
            devCfg.append((address, devFunc, serNum, GenPnl(Toplevel())))
            devCon.fnClose()
        else: pass

#####
# This section is for Testing & Demonstration purposes,
# Comment this section out when connected to an active system
#####
#print 'Adding Test Entries to DevCfg'
##devCfg.append((2, 'Inverter Charger', 12345678, InvChg(Toplevel())))
##devCfg.append((2, 'Battery', 00000000, Battery(Toplevel())))
##devCfg.append((3, 'Charger', 23456789, SolChg(Toplevel())))
##devCfg.append((4, 'AGS', 34567890, GenPnl(Toplevel())))
##devCfg.append((5, 'Charger', 87654321, SolChg(Toplevel())))
##devCfg.append((6, 'User Interface', 98765432, ScPnl(Toplevel())))
##XBMonitor.setAddress(2)
#####
# End Testing Section
#####

```

```
# Create and start the Device Update Threads, Device Windows, and Device Menu
if devCfg:
```

```
    for i in range(0,len(devCfg)):
        devCfg[i][3].winfo_toplevel().title(devCfg[i][1] + ' (esc to close)')
        devCfg[i][3].winfo_toplevel().iconbitmap('Coda.ico')
        devCfg[i][3].winfo_toplevel().withdraw()
        def escKeyHandler(event, self=devCfg[i][3]):
            self.noEscape()
        devCfg[i][3].winfo_toplevel().bind('<Key-Escape>', escKeyHandler)
        devCfg[i][3].winfo_toplevel().protocol('WM_DELETE_WINDOW', lambda:0)
        devCfg[i][3].winfo_toplevel().resizable(width=False, height=False)
        deviceMenu.add_command(label=devCfg[i][1] + ':' + '%s' % devCfg[i][0], command=devCfg[i][3].winfo_toplevel().deiconify)
        devCfg[i][3].setNameCfg(devCfg[i][2], devCfg[i][0])
        thread.start_new(devCfg[i][3].worker,())
    else: pass
```

```
#####
#           END SYSTEM DEVICE DISCOVERY
#####
```

```
# Add the menus to the main status window
XBMonitor.master.config(menu=menubar,bg='#555588')
```

```
# Create and add a status bar showing the CanBus version to the main system window
version = StringVar()
version.set('Bus API Version: %s' % busCon.version)
Label(XBMonitor.master,bg='#D0D0BA',fg='black',textvariable=version,relief=SUNKEN).grid(row=3,column=0,columnspan=3,sticky=E+W,padx=2,pady=2)
```

```
# Start the Main Application Loop
XBMonitor.mainloop()
```

```

#####
#
#   File: Devices.py               Description: Classes for Xantrex XW Xanbus devices
#   Date: April 2008
#   Author: Cary Humphries        Assumptions: This program was written for an Apex
#   email: cary_humphries@hotmail.com    USBCAN dongle connected to a Xanbus network.
#
#####

#####
#   CLASS DESCRIPTIONS
#####
#   AcInput    => AC Line 1 & 2
#   AcOutput    => AC Load
#   BattState   => Battery State
#   InvState    => Inverter State
#   InvChg      => Inverter/Charger Device Window
#   Battery     => Battery Storage Device Window
#   SolChg      => Solar Charge Controller Device Window
#   GenPnl      => Generator Control Device Window
#   ScPnl       => System Control Panel Device Window
#####

from Tkinter import *
import PyXanBus, thread, time, tkFont
from ScrolledText import ScrolledText

#####
#   Main Status Display Panel Classes
#####

#####
#   AcInput    => AC Line 1 & 2 Main Status Display
#####
class AcInput(LabelFrame):
    def __init__(self, master=None):
        LblFrameFont = tkFont.Font(family='arial', size=9)
        LabelFrame.__init__(self, master, bg='#555588', fg='#EEEEEE', font=LblFrameFont, text='AC Input')
        self.grid(sticky=N+S+E+W)
        # column config
        self.columnconfigure(0, minsize=75, weight=1)
        self.columnconfigure(1, minsize=75, weight=1)
        self.columnconfigure(2, minsize=75, weight=1)
        # AcInput Instance Variables
        self.line1 = StringVar()
        self.line1Source = StringVar()
        self.line1volts = DoubleVar()
        self.line1amps = DoubleVar()
        self.line1watts = DoubleVar()
        self.line1hz = DoubleVar()
        self.line2 = StringVar()
        self.line2Source = StringVar()
        self.line2volts = DoubleVar()
        self.line2amps = DoubleVar()
        self.line2watts = DoubleVar()
        self.line2hz = DoubleVar()
        self.Addr = StringVar()
        self.buildWindow()
        thread.start_new(self.worker, ())

    def buildWindow(self):
        # AcInput Instance Variable default startup values
        self.setLineData('Line 1', 'Invalid', 0.0, 0.0, 0.0, 0.0)
        self.setLineData('Line 2', 'Invalid', 0.0, 0.0, 0.0, 0.0)
        self.setAddr(0)
        # Build the display
        LblDisplayFont = tkFont.Font(family='arial', size=9)
        # Line 1 data values
        self.LblLine1 =
Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.line1Source)
        self.LblLine1Volts =
Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.line1volts)
        self.LblLine1Amps =
Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.line1amps)

```

```

        self.LblLine1Watts =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.line1watts)
        self.LblLine1Hz = Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.line1hz)
        # Line 2 data values
        self.LblLine2 =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.line2Source)
        self.LblLine2Volts =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.line2volts)
        self.LblLine2Amps =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.line2amps)
        self.LblLine2Watts =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.line2watts)
        self.LblLine2Hz = Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.line2hz)
        # Data tags
        self.LblAcVolts = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Volts')
        self.LblAcAmps = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Amps')
        self.LblAcWatts = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Watts')
        self.LblAcHz = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Hertz')
        # Grid the data labels
        self.LblLine1.grid(row=0,column=0,sticky=N+E+W)
        Label(self,bg='#555588',text='').grid(row=0,column=1,sticky=N+E+W)
        self.LblLine2.grid(row=0,column=2,sticky=N+E+W)
        self.LblLine1Volts.grid(row=1,column=0,sticky=N+E+W)
        self.LblAcVolts.grid(row=1,column=1,sticky=N+E+W)
        self.LblLine2Volts.grid(row=1,column=2,sticky=N+E+W)
        self.LblLine1Amps.grid(row=2,column=0,sticky=N+E+W)
        self.LblAcAmps.grid(row=2,column=1,sticky=N+E+W)
        self.LblLine2Amps.grid(row=2,column=2,sticky=N+E+W)
        self.LblLine1Watts.grid(row=3,column=0,sticky=N+E+W)
        self.LblAcWatts.grid(row=3,column=1,sticky=N+E+W)
        self.LblLine2Watts.grid(row=3,column=2,sticky=N+E+W)
        self.LblLine1Hz.grid(row=4,column=0,sticky=N+E+W)
        self.LblAcHz.grid(row=4,column=1,sticky=N+E+W)
        self.LblLine2Hz.grid(row=4,column=2,sticky=N+E+W)

# Method to update the StringVar data displays for AC Input
def setLineData(self,line,source,volts,amps,watts,hertz):
    if line == 'Line 1':
        self.line1.set(line)
        self.line1Source.set(source)
        self.line1volts.set(volts)
        self.line1amps.set(amps)
        self.line1watts.set(watts)
        self.line1hz.set(hertz)
    elif line == 'Line 2':
        self.line2.set(line)
        self.line2Source.set(source)
        self.line2volts.set(volts)
        self.line2amps.set(amps)
        self.line2watts.set(watts)
        self.line2hz.set(hertz)
    else:
        pass

def setAddr(self,a):
    self.Addr.set(a)

# Data updater threaded function
def worker(self):
    while True:
        if self.Addr.get() != 0:
            try:
                devCon = PyXanBus.clDeviceConnection(busCon, self.Addr.get(), 0)
                message = devCon.fnRequest(0x1F003)
                setLineData(message[0].fnGetValue('Line'),message[0].fnGetValue('AcSrcId'), \
                    message[0].fnGetValue('Voltage'),message[0].fnGetValue('Current'), \
                    message[0].fnGetValue('RealPwr'),message[0].fnGetValue('Freq'))
                setLineData(message[1].fnGetValue('Line'),message[1].fnGetValue('AcSrcId'), \
                    message[1].fnGetValue('Voltage'),message[1].fnGetValue('Current'), \
                    message[1].fnGetValue('RealPwr'),message[1].fnGetValue('Freq'))
                devCon.fnClose()
            except: pass
        time.sleep(5)

```

```

#####
# AcOutput => AC Load Main Status Display
#####
class AcOutput(LabelFrame):
    def __init__(self, master=None):
        LblFrameFont = tkFont.Font(family='arial', size=9)
        LabelFrame.__init__(self, master, bg='#555588', fg='#EEEEEE', font=LblFrameFont, text='AC Output')
        self.grid(sticky=N+S+E+W)
        self.columnconfigure(0, minsize=100, weight=1)
        self.columnconfigure(1, minsize=100, weight=1)
        # AcOutput Instance Variables
        self.load = StringVar()
        self.volts = DoubleVar()
        self.amps = DoubleVar()
        self.watts = DoubleVar()
        self.freq = DoubleVar()
        self.Addr = StringVar()
        self.buildWindow()

    def buildWindow(self):
        # AcOutput Instance Variable default startup values
        self.setLoadData('AC Load 1', 0.0, 0.0, 0.0, 0.0)
        self.setAddr(0)
        # Build the display
        LblDisplayFont = tkFont.Font(family='arial', size=9)
        # AC Output\Load data values
        self.LblLoad = Label(self, bg='#555588', fg='ffffff', font=LblDisplayFont, textvariable=self.load)
        self.LblVolts = Label(self, bg='#555588', fg='ff6600', font=LblDisplayFont, text='Volts')
        self.LblVoltsData = Label(self, bg='#555588', fg='ffffff', font=LblDisplayFont, textvariable=self.volts)
        self.LblAmps = Label(self, bg='#555588', fg='ff6600', font=LblDisplayFont, text='Amps')
        self.LblAmpsData = Label(self, bg='#555588', fg='ffffff', font=LblDisplayFont, textvariable=self.amps)
        self.LblWatts = Label(self, bg='#555588', fg='ff6600', font=LblDisplayFont, text='Watts')
        self.LblWattsData = Label(self, bg='#555588', fg='ffffff', font=LblDisplayFont, textvariable=self.watts)
        self.LblFreq = Label(self, bg='#555588', fg='ff6600', font=LblDisplayFont, text='Hertz')
        self.LblFreqData = Label(self, bg='#555588', fg='ffffff', font=LblDisplayFont, textvariable=self.freq)
        # Grid the data labels
        self.LblLoad.grid(row=0, column=0, columnspan=2, sticky=N+E+W)
        self.LblVolts.grid(row=1, column=0, sticky=N+E+W)
        self.LblVoltsData.grid(row=1, column=1, sticky=N+E+W)
        self.LblAmps.grid(row=2, column=0, sticky=N+E+W)
        self.LblAmpsData.grid(row=2, column=1, sticky=N+E+W)
        self.LblWatts.grid(row=3, column=0, sticky=N+E+W)
        self.LblWattsData.grid(row=3, column=1, sticky=N+E+W)
        self.LblFreq.grid(row=4, column=0, sticky=N+S+E+W)
        self.LblFreqData.grid(row=4, column=1, sticky=N+S+E+W)

    def setLoadData(self, source, volts, amps, watts, hertz):
        self.load.set(source)
        self.volts.set(volts)
        self.amps.set(amps)
        self.watts.set(watts)
        self.freq.set(hertz)

    def setAddr(self, a):
        self.Addr.set(a)

    # Data updater threaded function
    def worker(self):
        while True:
            if self.Addr.get() != 0:
                try:
                    devCon = PyXanBus.clDeviceConnection(busCon, self.Addr.get(), 0)
                    message = devCon.fnRequest(0x1F006)
                    setLoadData(message[0].fnGetValue('AcSrcId'), message[0].fnGetValue('Voltage'), \
                                message[0].fnGetValue('Current'), message[0].fnGetValue('RealPwr'), \
                                message[0].fnGetValue('Freq'))
                    devCon.fnClose()

```

```

        except: pass
        time.sleep(5)

```

```

#####
# BattState => Battery State Main Status Display
#####
class BattState(LabelFrame):
    def __init__(self, master=None):
        LblFrameFont = tkFont.Font(family='arial', size=9)
        LabelFrame.__init__(self, master, bg='#555588', fg='#EEEEEE', font=LblFrameFont, text='Battery')
        self.grid(sticky=N+S+E+W)
        # column config
        self.columnconfigure(0, minsize=100, weight=1)
        self.columnconfigure(1, minsize=100, weight=1)
        self.columnconfigure(2, minsize=100, weight=1)
        self.columnconfigure(3, minsize=100, weight=1)
        self.columnconfigure(4, minsize=100, weight=1)
        self.columnconfigure(5, minsize=100, weight=1)
        # Battery Status Instance Variables
        self.battVolts = DoubleVar() # 0x1F0C4 'Voltage'
        self.battAmps = DoubleVar() # 0x1F0C4 'Current'
        self.battWatts = DoubleVar() # 0x1F0C4 'Pwr'
        self.battTemp = DoubleVar() # 0x1F0C4 'Temp'
        self.battCap = DoubleVar() # 0x1F0C4 'BattCapPer'
        self.battWattHRem = DoubleVar() # 0x1F0C4 'EnergyRemain'
        self.Addr = StringVar()
        self.buildWindow()

    def buildWindow(self):
        # Battery Status Instance Variable default startup values
        self.setBattData(0.0, 0.0, 0.0, 0.0, 0.0, 0.0)
        self.setAddr(0)
        # Build the display
        LblDisplayFont = tkFont.Font(family='arial', size=9)
        # AC Output\Load data values
        self.LblBattVolts =
Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.battVolts)
        self.LblBattAmps =
Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.battAmps)
        self.LblBattWatts =
Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.battWatts)
        self.LblBattTemp =
Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.battTemp)
        self.LblBattCap = Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.battCap)
        self.LblBattWattHRem =
Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.battWattHRem)
        # Data tags
        self.LblVolts = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Volts')
        self.LblAmps = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Amps')
        self.LblWatts = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Watts')
        self.LblTemp = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Temp (F)')
        self.LblCap = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Capacity')
        self.LblWhRem = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Wh Remaining')
        # Grid the data labels
        self.LblVolts.grid(row=0, column=0, sticky=N+E+W)
        self.LblAmps.grid(row=0, column=1, sticky=N+E+W)
        self.LblWatts.grid(row=0, column=2, sticky=N+E+W)
        self.LblTemp.grid(row=0, column=3, sticky=N+E+W)
        self.LblCap.grid(row=0, column=4, sticky=N+E+W)
        self.LblWhRem.grid(row=0, column=5, sticky=N+E+W)
        self.LblBattVolts.grid(row=1, column=0, sticky=N+E+W)
        self.LblBattAmps.grid(row=1, column=1, sticky=N+E+W)
        self.LblBattWatts.grid(row=1, column=2, sticky=N+E+W)
        self.LblBattTemp.grid(row=1, column=3, sticky=N+E+W)
        self.LblBattCap.grid(row=1, column=4, sticky=N+E+W)
        self.LblBattWattHRem.grid(row=1, column=5, sticky=N+E+W)

    def setBattData(self, volts, amps, watts, temp, cap, wattH):
        self.battVolts.set(volts)
        self.battAmps.set(amps)

```

```

        self.battWatts.set(watts)
        self.battTemp.set(temp)
        self.battCap.set(cap)
        self.battWattHRem.set(wattH)

def setAddr(self,a):
    self.Addr.set(a)

# Data updater threaded function
def worker(self):
    while True:
        if self.Addr.get() != 0:
            try:
                devCon = PyXanBus.clDeviceConnection(busCon, self.Addr.get(), 0)
                message = devCon.fnRequest(0x1F0C4)
                setBattData(message[0].fnGetValue('Voltage'),message[0].fnGetValue('Current'), \
                    message[0].fnGetValue('Pwr'),message[0].fnGetValue('Temp'), \
                    message[0].fnGetValue('BattCapPer'),message[0].fnGetValue('EnergyRemain'))
                devCon.fnClose()
            except: pass
            time.sleep(5)

#####
# InvState => Inverter State Main Status Display
#####
class InvState(LabelFrame):
    def __init__(self,master=None):
        LblFrameFont = tkFont.Font(family='arial',size=9)
        LabelFrame.__init__(self,master,bg='#555588',fg='#EEEEEE',font=LblFrameFont,text='Inverter')
        self.grid(sticky=N+S+E+W)
        # column config
        self.columnconfigure(0,minsize=100,weight=1)
        # Inverter Instance Variables
        self.invOpState = StringVar()
        self.invOpMode = StringVar()
        self.Addr = StringVar()
        self.buildWindow()

def buildWindow(self):
    # Inverter Instance Variable default startup values
    self.setInvData('Load Sense','Invalid')
    self.setAddr(0)
    # Build the display
    LblDisplayFont = tkFont.Font(family='arial', size=9)
    # Inverter data values
    self.LblOpState =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.invOpState,wraplength=100)
    self.LblOpMode =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.invOpMode,wraplength=100)
    # Data tags
    self.LblState = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='State')
    self.LblMode = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Mode')
    # Grid the data labels
    self.LblState.grid(row=0,column=0,sticky=N+E+W)
    self.LblOpState.grid(row=1,column=0,sticky=N+E+W)
    self.LblMode.grid(row=2,column=0,sticky=N+E+W)
    self.LblOpMode.grid(row=3,column=0,sticky=N+E+W)

def setInvData(self,state,mode):
    self.invOpState.set(state)
    self.invOpMode.set(mode)

def setAddr(self,a):
    self.Addr.set(a)

# Data updater threaded function
def worker(self):
    while True:
        if self.Addr.get() != 0:
            try:
                devCon = PyXanBus.clDeviceConnection(busCon, self.Addr.get(), 0)
                message = devCon.fnRequest(0x1F0BD)
                setInvData(message[0].fnGetValue('OpState'),message[0].fnGetValue('InvMode'))
                devCon.fnClose()
            except: pass
            time.sleep(5)

```

```

#####
#                               Individual Device Status Panel Classes                               #
#####
# Inverter/Charger Device Information to Display
#
# ModelName                      Inverter Operating State          Charger DC Source
# Unique Number                 Inverter Enabled/Disabled        Charger OpState
# AC Transfer Switch Mode       Inverter Mode              Charger Mode
# AC Transfer Switch OpState    Grid Enabled/disabled      Charger Enabled/Disabled
#                               Sell to Grid Enabled/disabled      Charging Algorithm
#                               Control Voltage Vdc
#                               Control Current A
#                               Control Current % max
#
#                               AC Input                          AC Output                          DC In/Out
# Line1                         Line2                         Line(SourceId)                   Line(SourceId)
# Voltage Vac                   Voltage Vac                 Voltage Vac                      Volt +/- Vdc
# Current A                     Current A                 Current A                       Current +/- A
# Frequency Hz                  Frequency Hz              Current % of max                 Power +/- W
# Quality                       Quality                     Frequency Hz                    Ripple Voltage Vac
# Apmaster Power Ah            Apmaster Power Ah        Apmaster Power Ah
# Real Power W                 Real Power W              Real Power W
#
# Fault Log                     Warning Log
# String Id (Seq #)             String Id (Seq #)
# Fault Type                    Warning Type
# Fault Identifier              Warning Identifier
#
#####
# PGN's associated with this class:                               Functions for PGN
# 0x1F014(ProdInfoSts)PGN#126996 => setProdInfoSts()
# 0x13A00(NAMECfG)PGN#80384      => setNameCfG()
# 0x1F0BD(InvSts2)PGN#127165    => setInvSts2()
# 0x1F00E(ChgSts)PGN#126990     => setChgSts()
# 0x1F0C5(DcSrcSts2)PGN#127173  => setDcSrcSts2()
# 0x1F0BF(AcXferSwSts)PGN#127167 => setAcXferSwSts()
# 0x1F003(AcInStsRms)PGN#126979 => setAcInStsRms()
# 0x1F006(AcOutStsRms)PGN#126982 => setAcOutStsRms()
# 0x1F0C2(InvStatsAc2)PGN#127170 => setOutput()
#
#####
# InvChg => Inverter/Charger Device Window
class InvChg(LabelFrame):
    def __init__(self, master=None):
        LblFrameFont = tkFont.Font(family='arial', size=9)
        w =
LabelFrame.__init__(self, master, bg='#555588', fg='#EEEEEE', font=LblFrameFont, text='Inverter/Charger')
        self.grid(sticky=N+S+E+W)

        # Inverter/Charger Device Instance Variables
        # Device Info
        self.ModelName = StringVar() # 0x1F014 'ModelName'
0x1F014(ProdInfoSts)PGN#126996 # 0x1F014 'Fga' Finished goods assembly
        self.Fga = StringVar() # 0x13A00 'UniqueNumber' 0x13A00(NAMECfG)PGN#80384
        self.SerNo = StringVar()
        # AC Transfer Switch Status
0x1F0BF(AcXferSwSts)PGN#127167 # 0x1F0BF 'OpState' Which AC input is active
        self.XferOpState = StringVar() # 0x1F0BF 'XferSwMode' Which has priority?
        self.XferSwMode = StringVar()
        # Inverter Status (AC/DC)=>AC

```

```

self.InvAcSource = StringVar()
self.InvOpState = StringVar()
self.InvMode = StringVar()
self.InvEnabled = StringVar()
self.InvGridEn = StringVar()
self.InvSellGridEn = StringVar()
self.InvForceSell = StringVar()
# Charger Status (AC/DC)=>DC
self.ChgDcSource = StringVar()
self.ChgOpState = StringVar()
self.ChgOpMode = StringVar()
self.ChgEnabled = StringVar()
self.ChgOpAlgorithm = StringVar()
self.ChgOpCntlVdc = DoubleVar()
self.ChgOpCntlAdc = DoubleVar()
self.ChgOpCntlAdcPerMax = DoubleVar()
# DC Input
self.DCSource = StringVar()
self.DCVolts = DoubleVar()
self.DCAmps = DoubleVar()
self.DCWatts = DoubleVar()
# AC Input Line 1
self.Line1 = StringVar()
self.Line1SourceId = StringVar()
self.Line1Quality = StringVar()
self.Line1Volt = DoubleVar()
self.Line1Amp = DoubleVar()
self.Line1Freq = DoubleVar()
self.Line1AppPwr = DoubleVar()
self.Line1RealPwr = DoubleVar()
# AC Input Line 2
self.Line2 = StringVar()
self.Line2SourceId = StringVar()
self.Line2Quality = StringVar()
self.Line2Volt = DoubleVar()
self.Line2Amp = DoubleVar()
self.Line2Freq = DoubleVar()
self.Line2AppPwr = DoubleVar()
self.Line2RealPwr = DoubleVar()
# AC Output (Load)
self.LoadSourceId = StringVar()
self.LoadVolt = DoubleVar()
self.LoadAmp = DoubleVar()
self.LoadAmpPerMax = DoubleVar()
self.LoadFreq = DoubleVar()
self.LoadWaveForm = StringVar()
self.LoadAppPwr = DoubleVar()
self.LoadRealPwr = DoubleVar()
self.LoadRealPwrPerMax = DoubleVar()
# Watts Output
self.WattsOutput = DoubleVar()
self.Address = IntVar()
self.buildWindow()

def buildWindow(self):
    # Inverter/Charger Device Instance Variable default startup values
    self.setProdInfoSts('XW Inverter Charger', 'XW6048-120/240-60')
    self.setNameCfg('000000004321', 0)
    self.setInvSts2('Invalid', 'Engaging Inverter', 'Invalid', 'On', 'On', 'On', 'Off')
    self.setChgSts('Invalid', 'Not Charging', 'Standalone', 'On', 'Invalid', 0.0, 0.0, 0.0)
    self.setDcSrcSts2('Invalid', 0.0, 0.0, 0.0)
    self.setAcXferSwSts('Inactive', 'Unknown')
    self.setAcInStsRms('Line 1', 'Invalid', 'Invalid', 0.0, 0.0, 0.0, 0.0, 0.0)
    self.setAcInStsRms('Line 2', 'Invalid', 'Invalid', 0.0, 0.0, 0.0, 0.0, 0.0)
    self.setAcOutStsRms('Invalid', 0.0, 0.0, 0.0, 0.0, 'Invalid', 0.0, 0.0, 0.0)
    self.setOutput(0.0)
    # Build the display
    lblNameFont = tkFont.Font(family='arial', size=10)
    lblDisplayFont = tkFont.Font(family='arial', size=9)
    btnDisplayFont = tkFont.Font(family='arial', size=8)
    txtDisplayFont = tkFont.Font(family='arial', size=9)
    lblNoteFont = tkFont.Font(family='arial', size=8)
    # LabelFrame sections
    ChgFrame = LabelFrame(self, bg='#555588', fg='#EEEEEE', text='Charger')
    InvFrame = LabelFrame(self, bg='#555588', fg='#EEEEEE', text='Inverter')
    AC1Frame = LabelFrame(self, bg='#555588', fg='#EEEEEE', text='AC Line 1')

```

0x1F0BD 'AcSrcId' 0x1F0BD(InvSts2) PGN#127165

0x1F0BD 'OpState'

0x1F0BD 'InvMode'

0x1F0BD 'InvEnDis'

0x1F0BD 'GridEnDis'

0x1F0BD 'SellToGridEnDis'

0x1F0BD 'ForceSell' 0x1F00E(ChgSts) PGN#126990

0x1F00E 'DcSrcId'

0x1F00E 'OpState'

0x1F00E 'ChgMode'

0x1F00E 'ChgEnDis'

0x1F00E 'ChgAlgorithm'

0x1F00E 'CtrlV'

0x1F00E 'CtrlI'

0x1F00E 'CtrlIPer'

0x1F0C5 'DcSrcId' 0x1F0C5(DcSrcSts2) PGN#127173

0x1F0C5 'Voltage'

0x1F0C5 'Current'

0x1F0C5 'Pwr'

0x1F003 'Line' 0x1F003(AcInStsRms) PGN#126979

0x1F003 'AcSrcId'

0x1F003 'Quality'

0x1F003 'Voltage'

0x1F003 'Current'

0x1F003 'Freq'

0x1F003 'ApparentPwr'

0x1F003 'RealPwr'

0x1F003 'Line'

0x1F003 'AcSrcId'

0x1F003 'Quality'

0x1F003 'Voltage'

0x1F003 'Current'

0x1F003 'Freq'

0x1F003 'ApparentPwr'

0x1F003 'RealPwr'

0x1F006 'AcSrcId' 0x1F006(AcOutStsRms) PGN#126982

0x1F006 'Voltage'

0x1F006 'Current'

0x1F006 'CurrentPer'

0x1F006 'Freq'

0x1F006 'Waveform'

0x1F006 'ApparentPwr'

0x1F006 'RealPwr'

0x1F006 'RealPwrPer'

0x1F0C2(Inverter Statistics - AC 2) PGN#127170

0x1F0C2 'Energy' use when 'PreviousPeriod' == (4=Day)

```

AC2Frame = LabelFrame(self,bg='#555588',fg='#EEEEEE',text='AC Line 2')
ACOutFrame = LabelFrame(self,bg='#555588',fg='#EEEEEE',text='AC Load')
DCFrame = LabelFrame(self,bg='#555588',fg='#EEEEEE',text='DC Source')
ChgFrame.columnconfigure(0,minsize=150,weight=1)
InvFrame.columnconfigure(0,minsize=150,weight=1)
AC1Frame.columnconfigure(0,minsize=150,weight=1)
AC2Frame.columnconfigure(0,minsize=150,weight=1)
ACOutFrame.columnconfigure(0,minsize=150,weight=1)
DCFrame.columnconfigure(0,minsize=150,weight=1)
ChgFrame.columnconfigure(1,minsize=150,weight=1)
InvFrame.columnconfigure(1,minsize=150,weight=1)
AC1Frame.columnconfigure(1,minsize=150,weight=1)
AC2Frame.columnconfigure(1,minsize=150,weight=1)
ACOutFrame.columnconfigure(1,minsize=150,weight=1)
DCFrame.columnconfigure(1,minsize=150,weight=1)
# Data tags & value labels
self.LblModelName=Label(self,bg='#555588',fg='#ffffff',font=LblNameFont,textvariable=self.ModelName)
self.LblFga=Label(self,bg='#555588',fg='#ffffff',font=LblNameFont,textvariable=self.Fga)
self.LblSerNum=Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Unique ID')
self.LblSerNumData=Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.SerNo)
self.LblXferOpState=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Xfer Switch
State')
self.LblXferOpStateData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.XferOpState)
self.LblXferSwMode=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Xfer Switch
Mode')
self.LblXferSwModeData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.XferSwMode)
# Inverter data
self.LblInvAcSource=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Load')
self.LblInvAcSourceData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.InvAcSource)
self.LblInvOpState=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='State')
self.LblInvOpStateData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.InvOpState)
self.LblInvMode=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Mode')
self.LblInvModeData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.InvMode)
self.LblInvEnabled=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Enabled')
self.LblInvEnabledData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.InvEnabled)
self.LblInvGridEn=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Grid Enabled')
self.LblInvGridEnData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.InvGridEn)
self.LblInvSellGridEn=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Sell
Enabled')
self.LblInvSellGridEnData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.InvSellGridEn)
self.LblInvForceSell=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Force Sell')
self.LblInvForceSellData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.InvForceSell)
self.LblWattsOutput=Label(InvFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Energy Today
(Wh)')
self.LblWattsOutputData=
Label(InvFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.WattsOutput)
# Charger data
self.LblChgDcSource=Label(ChgFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Source')
self.LblChgDcSourceData=
Label(ChgFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.ChgDcSource)
self.LblChgOpState=Label(ChgFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='State')
self.LblChgOpStateData=
Label(ChgFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.ChgOpState)
self.LblChgOpMode=Label(ChgFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Mode')
self.LblChgOpModeData=
Label(ChgFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.ChgOpMode)
self.LblChgEnabled=Label(ChgFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Enabled')
self.LblChgEnabledData=
Label(ChgFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.ChgEnabled)
self.LblChgOpAlgorithm=Label(ChgFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Algorithm')
self.LblChgOpAlgorithmData=
Label(ChgFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.ChgOpAlgorithm)
self.LblChgOpCntlVdc=Label(ChgFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Control
Voltage')
self.LblChgOpCntlVdcData=
Label(ChgFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.ChgOpCntlVdc)

```

```

self.LblChgOpCntrlAdc=Label (ChgFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Control
Amps')
self.LblChgOpCntrlAdcData=
Label (ChgFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.ChgOpCntrlAdc)

self.LblChgOpCntrlAdcPerMax=Label (ChgFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Maximum
Amps (%)')
self.LblChgOpCntrlAdcPerMaxData=
Label (ChgFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.ChgOpCntrlAdcPerMax)
# DC source data
self.LblDCSource=Label (DCFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Source')
self.LblDCSourceData=
Label (DCFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.DCSource)
self.LblDCVolts=Label (DCFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Volts +/-')

self.LblDCVoltsData=Label (DCFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.DCVolts)
self.LblDCAmps=Label (DCFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Amps +/-')
self.LblDCAmpsData
=Label (DCFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.DCAmps)
self.LblDCWatts=Label (DCFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Watts +/-')

self.LblDCWattsData=Label (DCFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.DCWatts)
# AC input source data
self.LblLine1=Label (AC1Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Source 1')

self.LblLine1Data=Label (AC1Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line1)
self.LblLine1SourceId=Label (AC1Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Source Id')
self.LblLine1SourceIdData=
Label (AC1Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line1SourceId)
self.LblLine1Quality=Label (AC1Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Quality')
self.LblLine1QualityData=
Label (AC1Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line1Quality)
self.LblLine1Volt=Label (AC1Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Volts')
self.LblLine1VoltData=
Label (AC1Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line1Volt)
self.LblLine1Amp=Label (AC1Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Amps')
self.LblLine1AmpData=
Label (AC1Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line1Amp)
self.LblLine1Freq=Label (AC1Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Freq (Hz)')
self.LblLine1FreqData=
Label (AC1Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line1Freq)
self.LblLine1AppPwr=Label (AC1Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Apparent
Power (VA)')
self.LblLine1AppPwrData=
Label (AC1Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line1AppPwr)
self.LblLine1RealPwr=Label (AC1Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Real
Power (W)')
self.LblLine1RealPwrData=
Label (AC1Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line1RealPwr)
self.LblLine2=Label (AC2Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Source 2')

self.LblLine2Data=Label (AC2Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line2)
self.LblLine2SourceId=Label (AC2Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Source Id')
self.LblLine2SourceIdData=
Label (AC2Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line2SourceId)
self.LblLine2Quality=Label (AC2Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Quality')
self.LblLine2QualityData=
Label (AC2Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line2Quality)
self.LblLine2Volt=Label (AC2Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Volts')
self.LblLine2VoltData=
Label (AC2Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line2Volt)
self.LblLine2Amp=Label (AC2Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Amps')
self.LblLine2AmpData=
Label (AC2Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line2Amp)
self.LblLine2Freq=Label (AC2Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Freq (Hz)')
self.LblLine2FreqData=
Label (AC2Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line2Freq)
self.LblLine2AppPwr=Label (AC2Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Apparent
Power (VA)')
self.LblLine2AppPwrData=
Label (AC2Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line2AppPwr)
self.LblLine2RealPwr=Label (AC2Frame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Real
Power (W)')
self.LblLine2RealPwrData=
Label (AC2Frame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.Line2RealPwr)
# AC output (load) data

```

```

        self.LblLoadSourceId=Label(ACOutFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Output')
        self.LblLoadSourceIdData=
Label(ACOutFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.LoadSourceId)
        self.LblLoadVolt=Label(ACOutFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Output
Volts')
        self.LblLoadVoltData=
Label(ACOutFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.LoadVolt)
        self.LblLoadAmp=Label(ACOutFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Output Amps')
        self.LblLoadAmpData=
Label(ACOutFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.LoadAmp)
        self.LblLoadAmpPerMax=Label(ACOutFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Maximum
Amps(%)')
        self.LblLoadAmpPerMaxData=
Label(ACOutFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.LoadAmpPerMax)
        self.LblLoadFreq=Label(ACOutFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Output
Freq(Hz)')
        self.LblLoadFreqData=
Label(ACOutFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.LoadFreq)
        self.LblLoadWaveForm=Label(ACOutFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='AC Output
Waveform')
        self.LblLoadWaveFormData=
Label(ACOutFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.LoadWaveForm)
        self.LblLoadAppPwr=Label(ACOutFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Apparent
Power(VA)')
        self.LblLoadAppPwrData=
Label(ACOutFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.LoadAppPwr)
        self.LblLoadRealPwr=Label(ACOutFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Real
Power(W)')
        self.LblLoadRealPwrData=
Label(ACOutFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.LoadRealPwr)
        self.LblLoadRealPwrPerMax=Label(ACOutFrame,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Real
Power(%)')
        self.LblLoadRealPwrPerMaxData=
Label(ACOutFrame,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.LoadRealPwrPerMax)
        # Grid the data labels
        self.LblModelName.grid(row=0,column=0,padx=5,sticky=N+S+E)
        self.LblFga.grid(row=0,column=1,padx=5,sticky=N+S+W)
        self.LblSerNum.grid(row=1,column=0,padx=5,sticky=N+S+E)
        self.LblSerNumData.grid(row=1,column=1,padx=5,sticky=N+S+W)

Canvas(self,bg='gray',highlightthickness=0,height=2).grid(row=2,column=0,columnspan=4,padx=10,sticky=E+W)
        # Grid the data subframes
        InvFrame.grid(row=3,column=0,padx=5,pady=5,sticky=N+S+E+W)
        ChgFrame.grid(row=3,column=1,padx=5,pady=5,sticky=N+S+E+W)
        AC1Frame.grid(row=4,column=0,padx=5,pady=5,sticky=N+S+E+W)
        AC2Frame.grid(row=4,column=1,padx=5,pady=5,sticky=N+S+E+W)
        ACOutFrame.grid(row=5,column=0,padx=5,pady=5,sticky=N+S+E+W)
        DCFrame.grid(row=5,column=1,padx=5,pady=5,sticky=N+S+E+W)
        # Inverter subframe data
        self.LblInvAcSource.grid(row=0,column=0,sticky=N+S+W,padx=10)
        self.LblInvAcSourceData.grid(row=0,column=1,sticky=N+S+W,padx=10)
        self.LblInvOpState.grid(row=1,column=0,sticky=N+S+W,padx=10)
        self.LblInvOpStateData.grid(row=1,column=1,sticky=N+S+W,padx=10)
        self.LblInvMode.grid(row=2,column=0,sticky=N+S+W,padx=10)
        self.LblInvModeData.grid(row=2,column=1,sticky=N+S+W,padx=10)
        self.LblInvEnabled.grid(row=3,column=0,sticky=N+S+W,padx=10)
        self.LblInvEnabledData.grid(row=3,column=1,sticky=N+S+W,padx=10)
        self.LblInvGridEn.grid(row=4,column=0,sticky=N+S+W,padx=10)
        self.LblInvGridEnData.grid(row=4,column=1,sticky=N+S+W,padx=10)
        self.LblInvSellGridEn.grid(row=5,column=0,sticky=N+S+W,padx=10)
        self.LblInvSellGridEnData.grid(row=5,column=1,sticky=N+S+W,padx=10)
        self.LblInvForceSell.grid(row=6,column=0,sticky=N+S+W,padx=10)
        self.LblInvForceSellData.grid(row=6,column=1,sticky=N+S+W,padx=10)
        self.LblWattsOutput.grid(row=7,column=0,sticky=N+S+W,padx=10)
        self.LblWattsOutputData.grid(row=7,column=1,sticky=N+S+W,padx=10)
        # Charger subframe data
        self.LblChgDcSource.grid(row=0,column=0,sticky=N+S+W,padx=10)
        self.LblChgDcSourceData.grid(row=0,column=1,sticky=N+S+W,padx=10)
        self.LblChgOpState.grid(row=1,column=0,sticky=N+S+W,padx=10)
        self.LblChgOpStateData.grid(row=1,column=1,sticky=N+S+W,padx=10)
        self.LblChgOpMode.grid(row=2,column=0,sticky=N+S+W,padx=10)
        self.LblChgOpModeData.grid(row=2,column=1,sticky=N+S+W,padx=10)
        self.LblChgEnabled.grid(row=3,column=0,sticky=N+S+W,padx=10)
        self.LblChgEnabledData.grid(row=3,column=1,sticky=N+S+W,padx=10)
        self.LblChgOpAlgorithm.grid(row=4,column=0,sticky=N+S+W,padx=10)
        self.LblChgOpAlgorithmData.grid(row=4,column=1,sticky=N+S+W,padx=10)

```

```

self.LblChgOpCntlVdc.grid(row=5, column=0, sticky=N+S+W, padx=10)
self.LblChgOpCntlVdcData.grid(row=5, column=1, sticky=N+S+W, padx=10)
self.LblChgOpCntlAdc.grid(row=6, column=0, sticky=N+S+W, padx=10)
self.LblChgOpCntlAdcData.grid(row=6, column=1, sticky=N+S+W, padx=10)
self.LblChgOpCntlAdcPerMax.grid(row=7, column=0, sticky=N+S+W, padx=10)
self.LblChgOpCntlAdcPerMaxData.grid(row=7, column=1, sticky=N+S+W, padx=10)
# AC Input subframe data
self.LblLine1.grid(row=0, column=0, sticky=N+S+W, padx=10)
self.LblLine1Data.grid(row=0, column=1, sticky=N+S+W, padx=10)
self.LblLine1SourceId.grid(row=1, column=0, sticky=N+S+W, padx=10)
self.LblLine1SourceIdData.grid(row=1, column=1, sticky=N+S+W, padx=10)
self.LblLine1Quality.grid(row=2, column=0, sticky=N+S+W, padx=10)
self.LblLine1QualityData.grid(row=2, column=1, sticky=N+S+W, padx=10)
self.LblLine1Volt.grid(row=3, column=0, sticky=N+S+W, padx=10)
self.LblLine1VoltData.grid(row=3, column=1, sticky=N+S+W, padx=10)
self.LblLine1Amp.grid(row=4, column=0, sticky=N+S+W, padx=10)
self.LblLine1AmpData.grid(row=4, column=1, sticky=N+S+W, padx=10)
self.LblLine1Freq.grid(row=5, column=0, sticky=N+S+W, padx=10)
self.LblLine1FreqData.grid(row=5, column=1, sticky=N+S+W, padx=10)
self.LblLine1AppPwr.grid(row=6, column=0, sticky=N+S+W, padx=10)
self.LblLine1AppPwrData.grid(row=6, column=1, sticky=N+S+W, padx=10)
self.LblLine1RealPwr.grid(row=7, column=0, sticky=N+S+W, padx=10)
self.LblLine1RealPwrData.grid(row=7, column=1, sticky=N+S+W, padx=10)
self.LblLine2.grid(row=0, column=0, sticky=N+S+W, padx=10)
self.LblLine2Data.grid(row=0, column=1, sticky=N+S+W, padx=10)
self.LblLine2SourceId.grid(row=1, column=0, sticky=N+S+W, padx=10)
self.LblLine2SourceIdData.grid(row=1, column=1, sticky=N+S+W, padx=10)
self.LblLine2Quality.grid(row=2, column=0, sticky=N+S+W, padx=10)
self.LblLine2QualityData.grid(row=2, column=1, sticky=N+S+W, padx=10)
self.LblLine2Volt.grid(row=3, column=0, sticky=N+S+W, padx=10)
self.LblLine2VoltData.grid(row=3, column=1, sticky=N+S+W, padx=10)
self.LblLine2Amp.grid(row=4, column=0, sticky=N+S+W, padx=10)
self.LblLine2AmpData.grid(row=4, column=1, sticky=N+S+W, padx=10)
self.LblLine2Freq.grid(row=5, column=0, sticky=N+S+W, padx=10)
self.LblLine2FreqData.grid(row=5, column=1, sticky=N+S+W, padx=10)
self.LblLine2AppPwr.grid(row=6, column=0, sticky=N+S+W, padx=10)
self.LblLine2AppPwrData.grid(row=6, column=1, sticky=N+S+W, padx=10)
self.LblLine2RealPwr.grid(row=7, column=0, sticky=N+S+W, padx=10)
self.LblLine2RealPwrData.grid(row=7, column=1, sticky=N+S+W, padx=10)
# AC Load subframe data
self.LblLoadSourceId.grid(row=0, column=0, sticky=N+S+W, padx=10)
self.LblLoadSourceIdData.grid(row=0, column=1, sticky=N+S+W, padx=10)
self.LblLoadVolt.grid(row=1, column=0, sticky=N+S+W, padx=10)
self.LblLoadVoltData.grid(row=1, column=1, sticky=N+S+W, padx=10)
self.LblLoadAmp.grid(row=2, column=0, sticky=N+S+W, padx=10)
self.LblLoadAmpData.grid(row=2, column=1, sticky=N+S+W, padx=10)
self.LblLoadAmpPerMax.grid(row=3, column=0, sticky=N+S+W, padx=10)
self.LblLoadAmpPerMaxData.grid(row=3, column=1, sticky=N+S+W, padx=10)
self.LblLoadFreq.grid(row=4, column=0, sticky=N+S+W, padx=10)
self.LblLoadFreqData.grid(row=4, column=1, sticky=N+S+W, padx=10)
self.LblLoadWaveForm.grid(row=5, column=0, sticky=N+S+W, padx=10)
self.LblLoadWaveFormData.grid(row=5, column=1, sticky=N+S+W, padx=10)
self.LblLoadAppPwr.grid(row=6, column=0, sticky=N+S+W, padx=10)
self.LblLoadAppPwrData.grid(row=6, column=1, sticky=N+S+W, padx=10)
self.LblLoadRealPwr.grid(row=7, column=0, sticky=N+S+W, padx=10)
self.LblLoadRealPwrData.grid(row=7, column=1, sticky=N+S+W, padx=10)
self.LblLoadRealPwrPerMax.grid(row=8, column=0, sticky=N+S+W, padx=10)
self.LblLoadRealPwrPerMaxData.grid(row=8, column=1, sticky=N+S+W, padx=10)
# DC Input subframe data
self.LblDCSourceId.grid(row=0, column=0, sticky=N+W, padx=10)
self.LblDCSourceIdData.grid(row=0, column=1, sticky=N+W, padx=10)
self.LblDCVolts.grid(row=1, column=0, sticky=N+W, padx=10)
self.LblDCVoltsData.grid(row=1, column=1, sticky=N+W, padx=10)
self.LblDCAmps.grid(row=2, column=0, sticky=N+W, padx=10)
self.LblDCAmpsData.grid(row=2, column=1, sticky=N+W, padx=10)
self.LblDCWatts.grid(row=3, column=0, sticky=N+W, padx=10)
self.LblDCWattsData.grid(row=3, column=1, sticky=N+W, padx=10)
Label(DCFrame, bg='#555588', fg='FFFFFF', font=LblNoteFont, text='(Negative value indicates
charging) ').grid(row=4, column=0, columnspan=2, sticky=N+E+W)

# 0x1F014 (ProdInfoSts) PGN#126996
def setProdInfoSts(self, name, fga):
    self.ModelName.set(name)
    self.Fga.set(fga)

# 0x13A00 (NAMECfgr) PGN#80384

```

```

def setNameCfg(self, num, addr):
    self.SerNo.set(num)
    self.Address.set(addr)

# 0x1F0BD(InvSts2) PGN#127165
def setInvSts2(self, source, state, mode, en, grid, sell, force):
    self.InvAcSource.set(source)
    self.InvOpState.set(state)
    self.InvMode.set(mode)
    self.InvEnabled.set(en)
    self.InvGridEn.set(grid)
    self.InvSellGridEn.set(sell)
    self.InvForceSell.set(force)

# 0x1F00E(ChgSts) PGN#126990
def setChgSts(self, source, state, mode, en, algo, cntlvolt, cntlamp, ampper):
    self.ChgDcSource.set(source)
    self.ChgOpState.set(state)
    self.ChgOpMode.set(mode)
    self.ChgEnabled.set(en)
    self.ChgOpAlgorithm.set(algo)
    self.ChgOpCntlVdc.set(cntlvolt)
    self.ChgOpCntlAdc.set(cntlamp)
    self.ChgOpCntlAdcPerMax.set(ampper)

# 0x1F0C5(DcSrcSts2) PGN#127173
def setDcSrcSts2(self, source, volt, amp, watt):
    self.DCSource.set(source)
    self.DCVolts.set(volt)
    self.DCAmps.set(amp)
    self.DCWatts.set(watt)

# 0x1F0BF(AcXferSwSts) PGN#127167
def setAcXferSwSts(self, state, mode):
    self.XferOpState.set(state)
    self.XferSwMode.set(mode)

# 0x1F003(AcInStsRms) PGN#126979
def setAcInStsRms(self, line, source, qual, volt, amp, freq, apppwr, realpwr):
    try:
        if line == 'Line 1':
            self.Line1.set(line)
            self.Line1SourceId.set(source)
            self.Line1Quality.set(qual)
            self.Line1Volt.set(volt)
            self.Line1Amp.set(amp)
            self.Line1Freq.set(freq)
            self.Line1AppPwr.set(apppwr)
            self.Line1RealPwr.set(realpwr)
        elif line == 'Line 2':
            self.Line2.set(line)
            self.Line2SourceId.set(source)
            self.Line2Quality.set(qual)
            self.Line2Volt.set(volt)
            self.Line2Amp.set(amp)
            self.Line2Freq.set(freq)
            self.Line2AppPwr.set(apppwr)
            self.Line2RealPwr.set(realpwr)
        else:
            pass
    except: pass

# 0x1F006(AcOutStsRms) PGN#126982
def setAcOutStsRms(self, source, volt, amp, ampper, freq, wave, apppwr, realpwr, realper):
    self.LoadSourceId.set(source)
    self.LoadVolt.set(volt)
    self.LoadAmp.set(amp)
    self.LoadAmpPerMax.set(ampper)
    self.LoadFreq.set(freq)
    self.LoadWaveForm.set(wave)
    self.LoadAppPwr.set(apppwr)
    self.LoadRealPwr.set(realpwr)
    self.LoadRealPwrPerMax.set(realper)

# 0x1F0C2(InvStatsAc2) PGN#127170
def setOutput(self, wathh):

```

```

self.WattsOutput.set(watth)

# Hides the window without destroying the information
def noEscape(self):
    self.winfo_toplevel().withdraw()

# Thread function to keep the device updated
def worker(self):
    while True:
        try:
            devCon = PyXanBus.clDeviceConnection(busCon, self.Address.get(), 0)
            message = devCon.fnRequest(0x1F014)
            setProdInfoSts(message[0].fnGetValue('ModelName'),message[0].fnGetValue('Fga'))

            message = devCon.fnRequest(0x1F0BF)
            setAcXferSwSts(message[0].fnGetValue('OpState'),message[0].fnGetValue('XferSwMode'))

            message = devCon.fnRequest(0x1F0BD)
            setInvSts2(message[0].fnGetValue('AcSrcId'),message[0].fnGetValue('OpState'), \
                message[0].fnGetValue('InvMode'),message[0].fnGetValue('InvEnDis'), \
                message[0].fnGetValue('GridEnDis'),message[0].fnGetValue('SellToGridEnDis'), \
                message[0].fnGetValue('ForceSell'))

            message = devCon.fnRequest(0x1F00E)
            setChgSts(message[0].fnGetValue('DcSrcId'),message[0].fnGetValue('OpState'), \
                message[0].fnGetValue('ChgMode'),message[0].fnGetValue('ChgEnDis'), \
                message[0].fnGetValue('ChgAlgorithm'),message[0].fnGetValue('CtrlV'), \
                message[0].fnGetValue('CtrlI'),message[0].fnGetValue('CtrlIPer'))

            message = devCon.fnRequest(0x1F0C5)
            setDcSrcSts2(message[0].fnGetValue('DcSrcId'),message[0].fnGetValue('Voltage'), \
                message[0].fnGetValue('Current'),message[0].fnGetValue('Pwr'))

            message = devCon.fnRequest(0x1F003)
            setAcInStsRms(message[0].fnGetValue('Line'),message[0].fnGetValue('AcSrcId'), \
                message[0].fnGetValue('Quality'),message[0].fnGetValue('Voltage'), \
                message[0].fnGetValue('Current'),message[0].fnGetValue('Freq'), \
                message[0].fnGetValue('ApparentPwr'),message[0].fnGetValue('RealPwr'))
            setAcInStsRms(message[1].fnGetValue('Line'),message[1].fnGetValue('AcSrcId'), \
                message[1].fnGetValue('Quality'),message[1].fnGetValue('Voltage'), \
                message[1].fnGetValue('Current'),message[1].fnGetValue('Freq'), \
                message[1].fnGetValue('ApparentPwr'),message[1].fnGetValue('RealPwr'))

            message = devCon.fnRequest(0x1F006)
            setAcOutStsRms(message[0].fnGetValue('AcSrcId'),message[0].fnGetValue('Voltage'), \
                message[0].fnGetValue('Current'),message[0].fnGetValue('CurrentPer'), \
                message[0].fnGetValue('Freq'),message[0].fnGetValue('Waveform'), \
                message[0].fnGetValue('ApparentPwr'),message[0].fnGetValue('RealPwr'), \
                message[0].fnGetValue('RealPwrPer'))

            devCon.fnClose()
        except: pass
        time.sleep(5)

#####
# Battery Storage Device Information to Display
#
# DC Source          State of Charge          Time Remaining
# Voltage Vdc        State of Health          Energy Remaining
# Current A           Battery Capacity Remaining Time to Full
# Power W             Battery Capacity % max    Time since Full
# Temp C & F
# (returns Kelvin s.t. 273K = 0C)
#
#####

# Battery    => Battery Storage Device Window
class Battery(LabelFrame):
    def __init__(self, master=None):
        LblFrameFont = tkFont.Font(family='arial', size=9)
        LabelFrame.__init__(self, master, bg='#555588', fg='#EEEEEE', font=LblFrameFont, text='Battery Storage')
        self.grid(sticky=N+S+E+W)
        # column config
        self.columnconfigure(0, minsize=100, weight=1)
        self.columnconfigure(1, minsize=100, weight=1)
        self.columnconfigure(2, minsize=100, weight=1)

```

```

self.columnconfigure(3,minsize=100,weight=1)
# Battery Storage Device Instance Variables
self.batt = StringVar() # 0x1F0C4 'DcSrcId' 0x1F0C4(BattSts2) PGN#127172
self.battVolts = DoubleVar() # 0x1F0C4 'Voltage' Volts DC +/-
self.battAmps = DoubleVar() # 0x1F0C4 'Current' Amps
self.battWatts = DoubleVar() # 0x1F0C4 'Pwr' Watts
self.battTemp = DoubleVar() # 0x1F0C4 'Temp' Degrees K (273K = 0C)
self.battSoc = DoubleVar() # 0x1F0C4 'Soc' %
self.battSoh = DoubleVar() # 0x1F0C4 'Soh' %
self.battCap = DoubleVar() # 0x1F0C4 'BattCapRemain' AmpHours
self.battCapPer = DoubleVar() # 0x1F0C4 'BattCapPer' %
self.battTimeRem = DoubleVar() # 0x1F0C4 'TimeRemain' Minutes
self.battWattHRem = DoubleVar() # 0x1F0C4 'EnergyRemain' WattHours
self.battTimeToFull = DoubleVar() # 0x1F0C4 'TimeToFull' Minutes
self.battTimeSinceFull = DoubleVar() # 0x1F0C4 'TimeSinceFull' Minutes
self.Address = IntVar()
self.SerNo = StringVar()
self.buildWindow()

def buildWindow(self):
    # Battery Storage Device Instance Variable default startup values
    self.BattSts2('Invalid',0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0,0.0)
    # Build the display
    LblNameFont = tkFont.Font(family='arial',size=10)
    LblDisplayFont = tkFont.Font(family='arial',size=9)
    # AC Output\Load data values
    self.LblBatt = Label(self,bg='#555588',fg='#ffffff',font=LblNameFont,textvariable=self.batt)
    self.LblBattVolts =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battVolts)
    self.LblBattAmps =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battAmps)
    self.LblBattWatts =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battWatts)
    self.LblBattTemp =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battSoc)
    self.LblBattSoc = Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battSoh)
    self.LblBattSoh =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battWattHRem)
    self.LblBattCap = Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battCap)
    self.LblBattCapPer =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battCapPer)
    self.LblBattTimeRem =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battTimeRem)
    self.LblBattWattHRem =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battWattHRem)
    self.LblBattTimeTo =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battTimeToFull)
    self.LblBattTimeSince =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.battTimeSinceFull)
    # Data tags
    self.LblVolts = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Volts +/-')
    self.LblAmps = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Amps +/-')
    self.LblWatts = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Watts +/-')
    self.LblTemp = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Temp C/F')
    self.LblSoc = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='State of Charge')
    self.LblSoh = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='State of Health')
    self.LblCap = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Capacity Ah')
    self.LblCapPer = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Capacity %')
    self.LblTimeRem = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Time Remaining')
    self.LblWattHRem = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='WattHours
Remaining')
    self.LblTimeTo = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Time To Full')
    self.LblTimeSince = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Time Since Full')
    # Grid the data labels
    self.LblBatt.grid(row=0,column=0,columnspan=4,padx=10,sticky=N+E)
    #row 1
Canvas(self,bg='gray',highlightthickness=0,height=2).grid(row=1,column=0,columnspan=4,padx=10,sticky=E+W)
    #row 2
    self.LblVolts.grid(row=2,column=0,sticky=N+W,padx=10)
    self.LblBattVolts.grid(row=2,column=1,sticky=N+W,padx=10)
    self.LblSoc.grid(row=2,column=2,sticky=N+W,padx=10)
    self.LblBattSoc.grid(row=2,column=3,sticky=N+W,padx=10)
    #row 3
    self.LblAmps.grid(row=3,column=0,sticky=N+W,padx=10)
    self.LblBattAmps.grid(row=3,column=1,sticky=N+W,padx=10)

```

```

self.LblSoh.grid(row=3,column=2,sticky=N+W,padx=10)
self.LblBattSoh.grid(row=3,column=3,sticky=N+W,padx=10)
#row 4
self.LblWatts.grid(row=4,column=0,sticky=N+W,padx=10)
self.LblBattWatts.grid(row=4,column=1,sticky=N+W,padx=10)
self.LblTimeRem.grid(row=4,column=2,sticky=N+W,padx=10)
self.LblBattTimeRem.grid(row=4,column=3,sticky=N+W,padx=10)
#row 5
self.LblTemp.grid(row=5,column=0,sticky=N+W,padx=10)
self.LblBattTemp.grid(row=5,column=1,sticky=N+W,padx=10)
self.LblWattHRem.grid(row=5,column=2,sticky=N+W,padx=10)
self.LblBattWattHRem.grid(row=5,column=3,sticky=N+W,padx=10)
#row 6
self.LblCap.grid(row=6,column=0,sticky=N+W,padx=10)
self.LblBattCap.grid(row=6,column=1,sticky=N+W,padx=10)
self.LblTimeTo.grid(row=6,column=2,sticky=N+W,padx=10)
self.LblBattTimeTo.grid(row=6,column=3,sticky=N+W,padx=10)
#row 7
self.LblCapPer.grid(row=7,column=0,sticky=N+W,padx=10)
self.LblBattCapPer.grid(row=7,column=1,sticky=N+W,padx=10)
self.LblTimeSince.grid(row=7,column=2,sticky=N+W,padx=10)
self.LblBattTimeSince.grid(row=7,column=3,sticky=N+W,padx=10)

# 0x1F0C4(BattSts2) PGN#127172
def BattSts2(self,s,v,a,w,t,sc,sh,c,cp,tr,wr,tt,ts):
    self.batt.set(s)
    self.battVolts.set(v)
    self.battAmps.set(a)
    self.battWatts.set(w)
    self.battTemp.set(t)
    self.battSoc.set(sc)
    self.battSoh.set(sh)
    self.battCap.set(c)
    self.battCapPer.set(cp)
    self.battTimeRem.set(tr)
    self.battWattHRem.set(wr)
    self.battTimeToFull.set(tt)
    self.battTimeSinceFull.set(ts)

# 0x13A00(NAMECf) PGN#80384
def setNameCf(self,num,addr):
    self.SerNo.set(num)
    self.Address.set(addr)

# Hides the window without destroying the information
def noEscape(self):
    self.wininfo_toplevel().withdraw()

# Thread function to keep the device updated
def worker(self):
    while True:
        try:
            devCon = PyXanBus.clDeviceConnection(busCon, self.Address.get(), 0)
            message = devCon.fnRequest(0x1F0C4)
            BattSts2(message[0].fnGetValue('DcSrcId'),message[0].fnGetValue('Voltage'), \
                    message[0].fnGetValue('Current'),message[0].fnGetValue('Pwr'), \
                    message[0].fnGetValue('Temp'),message[0].fnGetValue('Soc'), \
                    message[0].fnGetValue('BattCapRemain'),message[0].fnGetValue('BattCapPer'), \
                    message[0].fnGetValue('TimeRemain'),message[0].fnGetValue('EnergyRemain'), \
                    message[0].fnGetValue('TimeToFull'),message[0].fnGetValue('TimeSinceFull'))
            devCon.fnClose()
        except: pass
        time.sleep(5)

#####
# Solar Charge Controller Device Information to Display
#
# Device Function          Charger DC Source          Control Voltage Vdc
# ModelName                Charger OpState          Control Current A
# Unique Number            Charger Mode              Control Current % max
#                           Charger Enabled/Disabled
#                           Charging Algorithm
#
# DC In\Out
# Line(DCSource)           Fault Log                  Warning Log
# Voltage +/- Vdc          String Id (Seq #)          String Id (Seq #)
# Current +/- A            Fault Type                  Warning Type

```

```

# Power +/- W          Fault Identifier      Warning Identifier
#
#####

# SolChg  => Solar Charge Controller Device Window
class SolChg(LabelFrame):
    def __init__(self, master=None):
        LblFrameFont = tkFont.Font(family='arial', size=9)
        LabelFrame.__init__(self, master, bg='#555588', fg='#EEEEEE', font=LblFrameFont, text='Solar Charge
Controller')
        self.grid(sticky=N+S+E+W)
        # column config
        self.columnconfigure(0, minsize=150, weight=1)
        self.columnconfigure(1, minsize=150, weight=1)
        self.columnconfigure(2, minsize=150, weight=1)
        self.columnconfigure(3, minsize=150, weight=1)
        # Solar Charger Device Instance Variables
        self.ModelName = StringVar()          # 0x1F014 'ModelName'          0x1F014 (ProdInfoSts)
PGN#126996
        self.Fga = StringVar()                # 0x1F014 'Fga'
        self.SerNo = StringVar()              # 0x13A00 'UniqueNumber'      0x13A00 (NAMECfg) PGN#80384
        self.DcSource = StringVar()           # 0x1F00E 'DcSrcId'          0x1F00E (ChgSts) PGN#126990
        self.OpState = StringVar()            # 0x1F00E 'OpState'
        self.OpMode = StringVar()              # 0x1F00E 'ChgMode'
        self.Enabled = StringVar()             # 0x1F00E 'ChgEnDis'
        self.Algorithm = StringVar()           # 0x1F00E 'ChgAlgorithm'
        self.CntlVdc = DoubleVar()             # 0x1F00E 'CtrlV'
        self.CntrlAdc = DoubleVar()            # 0x1F00E 'CtrlI'
        self.CntrlAdcPerMax = DoubleVar()      # 0x1F00E 'CtrlIPer'
        self.Vdc = DoubleVar()                 # 0x1F0C5 'Voltage'          0x1F0C5 (DcSrcSts2) PGN#127173
        self.Amps = DoubleVar()                # 0x1F0C5 'Current'
        self.Watts = DoubleVar()               # 0x1F0C5 'Pwr'
        self.Address = IntVar()
        self.buildWindow()

    def buildWindow(self):
        # Solar Charger Device Instance Variable default startup values
        # 0x1F014 (ProdInfoSts) PGN#126996
        self.setProdInfoSts('MPPT-60', 'XW-MPPT60-150')
        # 0x13A00 (NAMECfg) PGN#80384
        self.setNameCfg('XXXXXX009999', 0)
        # 0x1F00E (ChgSts) PGN#126990
        self.setChgSts('Invalid', 'Not Charging', 'Standalone', 'On', 'Invalid', 0.0, 0.0, 0.0)
        # 0x1F0C5 (DcSrcSts2) PGN#127173
        self.setDcSrcSts2(0.0, 0.0, 0.0)

        # Build the display
        LblNameFont=tkFont.Font(family='arial', size=10)
        LblDisplayFont=tkFont.Font(family='arial', size=9)
        TxtDisplayFont=tkFont.Font(family='arial', size=9)
        LblNoteFont=tkFont.Font(family='arial', size=8)
        # Data tags & value labels
        self.LblModelName=Label(self, bg='#555588', fg='#ffffff', font=LblNameFont, textvariable=self.ModelName)
        self.LblFga=Label(self, bg='#555588', fg='#ffffff', font=LblNameFont, textvariable=self.Fga)
        self.LblSerNum=Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='ID #')
        self.LblSerNumData=Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.SerNo)
        self.LblDcSource=Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='DC Source')

        self.LblDcSourceData=Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.DcSource)
        self.LblOpState=Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Current State')

        self.LblOpStateData=Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.OpState)
        self.LblOpMode=Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Current Mode')
        self.LblOpModeData=Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.OpMode)
        self.LblEnabled=Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Enabled')

        self.LblEnabledData=Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.Enabled)
        self.LblAlgorithm=Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Algorithm')

        self.LblAlgorithmData=Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.Algorithm)
        self.LblCntlVdc=Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Control Voltage')

        self.LblCntlVdcData=Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.CntlVdc)
        self.LblCntrlAdc=Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Control Amps')

        self.LblCntrlAdcData=Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.CntrlAdc)

```

```

self.LblCntrlAdcPerMax=Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Control Amps
%')
self.LblCntrlAdcPerMaxData=
Label(self,bg='#555588',fg='ffffff',font=LblDisplayFont,textvariable=self.CntrlAdcPerMax)
self.LblVdc=Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Volts +/-')
self.LblVdcData=Label(self,bg='#555588',fg='ffffff',font=LblDisplayFont,textvariable=self.Vdc)
self.LblAmps=Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='DC Amps +/-')
self.LblAmpsData=Label(self,bg='#555588',fg='ffffff',font=LblDisplayFont,textvariable=self.Amps)
self.LblWatts=Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Watts +/-')
self.LblWattsData=Label(self,bg='#555588',fg='ffffff',font=LblDisplayFont,textvariable=self.Watts)
# Grid the data labels
self.LblModelName.grid(row=0,column=0,columnspan=2,padx=10,sticky=N+S+E)
self.LblFga.grid(row=0,column=2,columnspan=2,padx=10,sticky=N+S+E)
#row 1

Canvas(self,bg='gray',highlightthickness=0,height=2).grid(row=1,column=0,columnspan=4,padx=10,sticky=E+W)
#row 2
self.LblSerNum.grid(row=2,column=0,sticky=N+S+W,padx=10)
self.LblSerNumData.grid(row=2,column=1,sticky=N+S+E+W,padx=10)
self.LblCntlVdc.grid(row=2,column=2,sticky=N+S+W,padx=10)
self.LblCntlVdcData.grid(row=2,column=3,sticky=N+S+E+W,padx=10)
#row 3
self.LblDcSource.grid(row=3,column=0,sticky=N+S+W,padx=10)
self.LblDcSourceData.grid(row=3,column=1,sticky=N+S+E+W,padx=10)
self.LblCntrlAdc.grid(row=3,column=2,sticky=N+S+W,padx=10)
self.LblCntrlAdcData.grid(row=3,column=3,sticky=N+S+E+W,padx=10)
#row 4
self.LblOpState.grid(row=4,column=0,sticky=N+S+W,padx=10)
self.LblOpStateData.grid(row=4,column=1,sticky=N+S+E+W,padx=10)
self.LblCntrlAdcPerMax.grid(row=4,column=2,sticky=N+S+W,padx=10)
self.LblCntrlAdcPerMaxData.grid(row=4,column=3,sticky=N+S+E+W,padx=10)
#row 5
self.LblOpMode.grid(row=5,column=0,sticky=N+S+W,padx=10)
self.LblOpModeData.grid(row=5,column=1,sticky=N+S+E+W,padx=10)
self.LblVdc.grid(row=5,column=2,sticky=N+S+W,padx=10)
self.LblVdcData.grid(row=5,column=3,sticky=N+S+E+W,padx=10)
#row 6
self.LblEnabled.grid(row=6,column=0,sticky=N+S+W,padx=10)
self.LblEnabledData.grid(row=6,column=1,sticky=N+S+E+W,padx=10)
self.LblAmps.grid(row=6,column=2,sticky=N+S+W,padx=10)
self.LblAmpsData.grid(row=6,column=3,sticky=N+S+E+W,padx=10)
#row 7
self.LblAlgorithm.grid(row=7,column=0,sticky=N+S+W,padx=10)
self.LblAlgorithmData.grid(row=7,column=1,sticky=N+S+E+W,padx=10)
self.LblWatts.grid(row=7,column=2,sticky=N+S+W,padx=10)
self.LblWattsData.grid(row=7,column=3,sticky=N+S+E+W,padx=10)

# 0x1F014(ProdInfoSts) PGN#126996
def setProdInfoSts(self,name,fga):
    self.ModelName.set(name)
    self.Fga.set(fga)

# 0x13A00(NAMECfgr) PGN#80384
def setNameCfgr(self,num,addr):
    self.SerNo.set(num)
    self.Address.set(addr)

# 0x1F00E(ChgSts) PGN#126990
def setChgSts(self,source,state,mode,en,algo,cntlvolt,cntlamp,ampper):
    self.DcSource.set(source)
    self.OpState.set(state)
    self.OpMode.set(mode)
    self.Enabled.set(en)
    self.Algorithm.set(algo)
    self.CntlVdc.set(cntlvolt)
    self.CntrlAdc.set(cntlamp)
    self.CntrlAdcPerMax.set(ampper)

# 0x1F0C5(DcSrcSts2) PGN#127173
def setDcSrcSts2(self,volt,amp,watt):
    self.Vdc.set(volt)
    self.Amps.set(amp)
    self.Watts.set(watt)

# Hides the window without destroying the information
def noEscape(self):

```

```

self.winfo_toplevel().withdraw()

# Thread function to keep the device updated
def worker(self):
    while True:
        try:
            devCon = PyXanBus.clDeviceConnection(busCon, self.Address.get(), 0)
            message = devCon.fnRequest(0x1F014)
            setProdInfoSts(message[0].fnGetValue('ModelName'), message[0].fnGetValue('Fga'))
            message = devCon.fnRequest(0x1F00E)
            setChgSts(message[0].fnGetValue('DcSrcId'), message[0].fnGetValue('OpState'), \
                    message[0].fnGetValue('ChgMode'), message[0].fnGetValue('ChgEnDis'), \
                    message[0].fnGetValue('ChgAlgorithm'), message[0].fnGetValue('CtrlV'), \
                    message[0].fnGetValue('CtrlI'), message[0].fnGetValue('CtrlIPer'))
            message = devCon.fnRequest(0x1F0C5)
            setDcSrcSts2(message[0].fnGetValue('Voltage'), message[0].fnGetValue('Current'), \
                    message[0].fnGetValue('Pwr'))
            devCon.fnClose()
        except: pass
        time.sleep(5)

#####
# AGS Device Information to Display (Automated Generator Start)
#
# ModelName          Generator State
# Unique Number      Generator ON Reason  -> Why Generator turned ON
# AC SourceId        Generator OFF Reason  -> Why Generator turned OFF
#                    AGS to Gen Signal    -> Current AGS signal to the Generator
#
#####

# GenPnl => Generator Control Device Window
class GenPnl(LabelFrame):
    def __init__(self, master=None):
        LblFrameFont = tkFont.Font(family='arial', size=9)
        LabelFrame.__init__(self, master, bg='#555588', fg='#EEEEEE', font=LblFrameFont, text='Generator Control
Panel')
        self.grid(sticky=N+S+E+W)
        # column config
        self.columnconfigure(0, minsize=100, weight=1)
        self.columnconfigure(1, minsize=100, weight=1)
        self.columnconfigure(2, minsize=100, weight=1)
        self.columnconfigure(3, minsize=100, weight=1)
        # AGS Device Instance Variables
        self.ModelName = StringVar()                # 0x1F014 'ModelName'                0x1F014(ProdInfoSts)
        PGN#126996
        self.Fga = StringVar()                      # 0x1F014 'Fga'
        self.SerNo = StringVar()                    # 0x13A00 'UniqueNumber'                0x13A00(NAMECfg) PGN#80384
        self.acSrc = StringVar()                    # 0x1F011 'AcSrcId'                    0x1F011(AgsSts) PGN#126993
        self.genOpState = StringVar()               # 0x1F001 'OpState'
        self.genState = StringVar()                 # 0x1F011 'GenState'
        self.genOnRsn = StringVar()                 # 0x1F011 'GenOnReason'
        self.genOffRsn = StringVar()                # 0x1F011 'GenOffReason'
        self.genSig = StringVar()                   # 0x1F011 'AgsGenSig'
        self.Address = IntVar()
        self.buildWindow()

    def buildWindow(self):
        # AGS Device Instance Variables default startup values
        self.setProdInfoSts('XW-AGS', 'XW Automatic Generator Start')
        self.setNameCfg('XXXX8675309', 0)
        self.setGenSts('Invalid', 'Not Operating', 'Stopped', 'Not On', 'Auto Off', 'Off')
        # Build the display
        LblNameFont = tkFont.Font(family='arial', size=10)
        LblDisplayFont = tkFont.Font(family='arial', size=9)
        TxtDisplayFont = tkFont.Font(family='arial', size=9)
        LblNoteFont = tkFont.Font(family='arial', size=8)
        # AGS data values
        self.LblModelName =
        Label(self, bg='#555588', fg='#ffffff', font=LblNameFont, textvariable=self.ModelName)
        self.LblFga = Label(self, bg='#555588', fg='#ffffff', font=LblNameFont, textvariable=self.Fga)
        self.LblSerNum = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='ID #')
        self.LblSerNumData =
        Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.SerNo)
        self.LblacSrc = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Source')
        self.LblacSrcData = Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.acSrc)

```

```

        self.LblgenOpState = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Generator Mode')
        self.LblgenOpStateData =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.genOpState)
        self.LblgenState = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Generator State')
        self.LblgenStateData =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.genState)
        self.LblgenOnRsn = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Reason On')
        self.LblgenOnRsnData =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.genOnRsn)
        self.LblgenOffRsn = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Reason Off')
        self.LblgenOffRsnData =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.genOffRsn)
        self.LblgenSig = Label(self,bg='#555588',fg='#ff6600',font=LblDisplayFont,text='Control Signal')
        self.LblgenSigData =
Label(self,bg='#555588',fg='#ffffff',font=LblDisplayFont,textvariable=self.genSig)
        # Grid the data labels
        #row 0-1
        self.LblModelName.grid(row=0,column=0,columnspan=2,sticky=N+E,padx=10)
        self.LblFga.grid(row=0,column=2,columnspan=2,sticky=N+W,padx=10)
        self.LblSerNum.grid(row=1,column=0,columnspan=3,sticky=N+E,padx=10)
        self.LblSerNumData.grid(row=1,column=3,sticky=N+E,padx=10)
        #row 2
Canvas(self,bg='gray',highlightthickness=0,height=2).grid(row=2,column=0,columnspan=4,padx=10,sticky=E+W)
        #row 3
        self.LblacSrc.grid(row=3,column=0,sticky=N+W,padx=10)
        self.LblacSrcData.grid(row=3,column=1,sticky=N+W,padx=10)
        self.LblgenSig.grid(row=3,column=2,sticky=N+W,padx=10)
        self.LblgenSigData.grid(row=3,column=3,sticky=N+W,padx=10)
        #row 3
        self.LblgenOpState.grid(row=4,column=0,sticky=N+W,padx=10)
        self.LblgenOpStateData.grid(row=4,column=1,sticky=N+W,padx=10)
        self.LblgenState.grid(row=4,column=2,sticky=N+W,padx=10)
        self.LblgenStateData.grid(row=4,column=3,sticky=N+W,padx=10)
        #row 5
        self.LblgenOnRsn.grid(row=5,column=0,sticky=N+W,padx=10)
        self.LblgenOnRsnData.grid(row=5,column=1,sticky=N+W,padx=10)
        self.LblgenOffRsn.grid(row=5,column=2,sticky=N+W,padx=10)
        self.LblgenOffRsnData.grid(row=5,column=3,sticky=N+W,padx=10)

# 0x1F014(ProdInfoSts) PGN#126996
def setProdInfoSts(self,name,fga):
    self.ModelName.set(name)
    self.Fga.set(fga)

# 0x13A00(NAMECfgr) PGN#80384
def setNameCfgr(self,num,addr):
    self.SerNo.set(num)
    self.Address.set(addr)

# 0x1F011(AgsSts) PGN#126993
def setGenSts(self,source,opstate,genstate,genon,genoff,sig):
    self.acSrc.set(source)
    self.genOpState.set(opstate)
    self.genState.set(genstate)
    self.genOnRsn.set(genon)
    self.genOffRsn.set(genoff)
    self.genSig.set(sig)

# Hides the window without destroying the information
def noEscape(self):
    self.winfo_toplevel().withdraw()

# Thread function to keep the device updated
def worker(self):
    while True:
        try:
            devCon = PyXanBus.clDeviceConnection(busCon, self.Address.get(), 0)
            message = devCon.fnRequest(0x1F014)
            setProdInfoSts(message[0].fnGetValue('ModelName'),message[0].fnGetValue('Fga'))
            message = devCon.fnRequest(0x1F011)
            setGenSts(message[0].fnGetValue('AcSrcId'),message[0].fnGetValue('OpState'), \
                message[0].fnGetValue('GenState'),message[0].fnGetValue('GenOnReason'), \
                message[0].fnGetValue('GenOffReason'),message[0].fnGetValue('AgsGenSig'))
            devCon.fnClose()
        except: pass

```

```

time.sleep(5)

#####
# SCP Device Information to Display (Sytem Control Panel)
#
# ModelName
# Unique Number
#
#####

# ScPnl => System Control Panel Device Window
class ScPnl(LabelFrame):
    def __init__(self, master=None):
        LblFrameFont = tkFont.Font(family='arial', size=9)
        LabelFrame.__init__(self, master, bg='#555588', fg='#EEEEEE', font=LblFrameFont, text='System Control
Panel')
        self.grid(sticky=N+S+E+W)
        # column config
        self.columnconfigure(0, minsize=150, weight=1)
        self.columnconfigure(1, minsize=150, weight=1)
        # SCP Device Instance Variables
        self.Model = StringVar() # 0x1F014 'ModelName' 0x1F014(ProdInfoSts) PGN#126996
        self.SerNo = StringVar() # 0x13A00 'UniqueNumber' 0x13A00(NAMECfgr) PGN#80384
        self.Address = IntVar()
        # SCP Device Instance Variables default startup values
        self.Model.set('SCP')
        self.SerNo.set('XXXX000001')
        # Build the display
        LblDisplayFont = tkFont.Font(family='arial', size=9)
        # SCP data values
        self.LblScpModel = Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.Model)
        self.LblScpSerNo = Label(self, bg='#555588', fg='#ffffff', font=LblDisplayFont, textvariable=self.SerNo)
        # SCP data tags
        self.Lblmodel = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='Model Name')
        self.LblSerNo = Label(self, bg='#555588', fg='#ff6600', font=LblDisplayFont, text='ID #')
        # Grid the data labels
        self.Lblmodel.grid(row=0, column=0, sticky=N+W, ipadx=10)
        self.LblScpModel.grid(row=0, column=1, sticky=N+W)
        self.LblSerNo.grid(row=1, column=0, sticky=N+W, ipadx=10)
        self.LblScpSerNo.grid(row=1, column=1, sticky=N+W)

# 0x13A00(NAMECfgr) PGN#80384
def setNameCfg(self, num, addr):
    self.SerNo.set(num)
    self.Address.set(addr)

# Hides the window without destroying the information
def noEscape(self):
    self.wininfo_toplevel().withdraw()

# Thread function to keep the device updated
def worker(self):
    while True:
        try:
            devCon = PyXanBus.clDeviceConnection(busCon, self.Address.get(), 0)
            message = devCon.fnRequest(0x1F014)
            self.Model.set(message[0].fnGetValue('ModelName'))
            devCon.fnClose()
        except: pass
        time.sleep(300)

```

```

#####
# Copyright (c) 2006 Xantrex International
#
# This file is the property of Xantrex International and shall not be reproduced,
# copied, or used as the basis for the manufacture or sale of equipment without
# the express written permission of Xantrex International.
#
# FILE NAME: PyXanBus.py
#
# PURPOSE:
#   This is the high level Python interface to XanBus. It ties in to both
#   the low level _xanbus extension module and the XanBus ODBC database
#   to provide access to interpreted messages on the network.
#
# FUNCTION(S):
#
#   local:
#
# NOTES:
#
# CHANGE HISTORY :
# $Log: PyXanBus.py $
#
# *****
# Revision: XanBusMain_TomL/28
# User: TomL      Date: 04/25/07   Time: 02:40AM
# Created a device connection method to get and validate an expected Ack
# message.
#
# *****
# Revision: XanBusMain_TomL/27
# User: TomL      Date: 04/12/07   Time: 02:19AM
# Allowed for multiple device connections to be open at once
#
# *****
# Revision: XanBusMain_TomL/26
# User: TomL      Date: 03/29/07   Time: 10:55PM
# Added an event monitor to check on bus events.
#
# Sped up devcon.fnGetString()
#
# *****
# Revision: XanBusMain_TomL/25
# User: TomL      Date: 03/10/07   Time: 02:43AM
# Unexpected messages received in ISORequest now ignored.
#
# *****
# Revision: XanBusMain_TomL/24
# User: TomL      Date: 02/16/07   Time: 12:40AM
# Upgraded to XanBus build 726
#
# *****
# Revision: XanBusMain_TomL/23
# User: TomL      Date: 02/15/07   Time: 11:30PM
# Enum selections now work fully with interpreted strings and clMessage.fnSetValue()
#
# *****
# Revision: XanBusMain_TomL/22
# User: TomL      Date: 02/14/07   Time: 12:29AM
# Minor code tidy
#
# *****
# Revision: XanBusMain_TomL/21
# User: TomL      Date: 02/10/07   Time: 02:53AM
# Message method fnGetFixedSignalMnems generalized to fnGetSignalMnems so
# that mnemonics can be retrieved for tuples as well.
#
# *****
# Revision: XanBusMain_TomL/20
# User: TomL      Date: 02/07/07   Time: 02:39AM
# Improved XML output using xml.dom.minidom
#
# Re-worked exceptions.
#
# *****

```

```

# Revision: XanBusMain_TomL/19
# User: TomL      Date: 01/24/07   Time: 12:26AM
# Added some intelligence to the collection of responses to ISORequest.
# Now much faster.

# *****
# Revision: XanBusMain_TomL/18
# User: TomL      Date: 01/19/07   Time: 01:16AM
# Added fnSetValue method to message class. This requires the message to
# take the database as a creation parameter.

# *****
# Revision: XanBusMain_TomL/17
# User: TomL      Date: 01/02/07   Time: 10:22PM
# Was checking key in the wrong dictionary

# *****
# Revision: XanBusMain_TomL/16
# User: TomL      Date: 12/08/06   Time: 02:35AM
# Fixed bad function call in NetIndic event callback

# *****
# Revision: XanBusMain_TomL/15
# User: TomL      Date: 12/04/06   Time: 09:50PM
# Removed print statements used for debugging callbacks

# *****
# Revision: XanBusMain_TomL/14
# User: TomL      Date: 12/02/06   Time: 01:34AM
# Attempted to make callbacks more crash-proof

# *****
# Revision: XanBusMain_TomL/13
# User: TomL      Date: 11/04/06   Time: 02:04AM
# Added enums for ClearCmd

# *****
# Revision: XanBusMain_TomL/12
# User: TomL      Date: 10/31/06   Time: 07:28PM
# Added more enums and some message manipulation methods

# *****
# Revision: XanBusMain_TomL/11
# User: TomL      Date: 10/25/06   Time: 11:02PM
# Can now create a message with a specified number of tuples.

# *****
# Revision: XanBusMain_TomL/10
# User: TomL      Date: 10/25/06   Time: 02:14AM
# Added clMessage.fnGetString() to get string from StringCfg messages

# Added function to get configuration limits

# *****
# Revision: XanBusMain_TomL/9
# User: TomL      Date: 10/21/06   Time: 02:36AM
# Can now set the raw value of a tuple field

# *****
# Revision: XanBusMain_TomL/8
# User: TomL      Date: 10/20/06   Time: 02:17AM
# Made sure that responses come through for gui forms

# *****
# Revision: XanBusMain_TomL/7
# User: TomL      Date: 10/06/06   Time: 05:19PM
# Now using SendEx, which tests result of sending.

# Added non-blocking send and receive functions for more responsive GUI applications.

# *****
# Revision: XanBusMain_TomL/6
# User: TomL      Date: 10/02/06   Time: 11:37PM
# Added more information about error response to ISO requests.

```

```

# Default timing for periodic transmit added as well.

# String method added for clMessage objects to facilitate printing

# *****
# Revision: XanBusMain_TomL/5
# User: TomL      Date: 09/29/06   Time: 01:25AM
# Prevented double closing of device connections

# *****
# Revision: XanBusMain_TomL/4
# User: TomL      Date: 09/28/06   Time: 01:22AM
# Added support for GUI, such as an event queue for callbacks, xml support
# for messages, and automatic support for broadcast status messages.

# *****
# Revision: XanBusMain_TomL/3
# User: TomL      Date: 09/22/06   Time: 01:41AM
# Added capability to create a new message from scratch.

# Added capability to receive all possible versions of strings sent to Diag
# terminal.

# *****
# Revision: XanBusMain_TomL/2
# User: TomL      Date: 09/20/06   Time: 01:38AM
# Code tidy and improved conformance to coding standard

# Added ComplexReq support to get StringCfg
#
#####

#####
#                               Includes
#####

import _xanbus
import threading
import time
import re
import PgDesc
import xml.dom.minidom

#####
#                               Constant Data
#####

XB_TICK_PERIOD = 0.01
XB_MAX_NODES = 10
MSG_TIMEOUT = 1.0 # seconds
POLL_PERIOD = 0.1
MAX_POLLS = int(MSG_TIMEOUT / POLL_PERIOD)

XB_eNETWORK_STATE_ERROR_ACTIVE      = 0
XB_eNETWORK_STATE_ERROR_PASSIVE     = 1
XB_eNETWORK_STATE_BUS_OFF           = 2
XB_eNETWORK_STATE_UNKNOWN           = 3

# Node change reasons
XB_eNODE_CHANGE_NEW_NODE      = 0 # Node has joined the bus
XB_eNODE_CHANGE_ADDRESS      = 1 # Existing node has a new address
XB_eNODE_CHANGE_BUMPED_OFF   = 2 # Node must give up its address
XB_eNODE_CHANGE_TIMED_OUT    = 3 # Node has not reported within timeout period

# Device function codes
XB_eFUNC_CODE_USER_INTERFACE      = 128
XB_eFUNC_CODE_INVERTER_CHARGER    = 129
XB_eFUNC_CODE_INVERTER            = 130
XB_eFUNC_CODE_CHARGER             = 131
XB_eFUNC_CODE_BATTERY_MONITOR     = 132
XB_eFUNC_CODE_AGS                 = 133
XB_eFUNC_CODE_DIAGNOSTIC_TERMINAL = 134
XB_eFUNC_CODE_GATEWAY             = 135

PGN_ADDR_GLOBAL                  = 255

```

```

# Commonly used PGNs
XB_PGN_ISO_ACK = 0x00e800
XB_PGN_ISO_ADDR_CLAIM = 0x00EE00
XB_PGN_STS = 0x01F00F

# Commonly used group functions
PGN_GRP_FUNC_DO_NOT_CARE = -1

# Commonly used enum values
XB_eCTRL_MODE_HIBERNATE = 0
XB_eCTRL_MODE_POWER_SAVE = 1
XB_eCTRL_MODE_SAFE = 2
XB_eCTRL_MODE_OPERATING = 3
XB_eCTRL_MODE_DIAGNOSTIC = 4

XB_eCMD_PAIR_OFF = 0
XB_eCMD_PAIR_ON = 1
XB_eCMD_PAIR_RESERVED = 2
XB_eCMD_PAIR_NO_CHANGE = 3

XB_eAGS_DESIRED_STATE_MANUAL_OFF = 0
XB_eAGS_DESIRED_STATE_MANUAL_ON = 1
XB_eAGS_DESIRED_STATE_AUTOMATIC = 2

XB_eSECONDARY_PWR_SUPPLY_DESIRED_STATE_MANUAL_OFF = 0
XB_eSECONDARY_PWR_SUPPLY_DESIRED_STATE_MANUAL_ON = 1
XB_eSECONDARY_PWR_SUPPLY_DESIRED_STATE_AUTOMATIC = 2

XB_eRST_CMD_REBOOT = 0
XB_eRST_CMD_REBOOT_DL_MODE = 1
XB_eRST_CMD_FACTORY = 2
XB_eRST_CMD_OEM = 3
XB_eRST_CMD_CLEAR_NV_AND_REBOOT = 4

XB_eFORCE_CHG_STATE_BULK = 1
XB_eFORCE_CHG_STATE_FLOAT = 2
XB_eFORCE_CHG_STATE_NO_FLOAT = 3

NUM_TESTS = 2

#####
#                               Class Definitions
#####

#####
#
# CLASS NAME: clBusConnection
#
# PURPOSE:
#   Connection object for using XanBus
#
# INPUTS:
#   DeviceFunction      - Device Function (ISO Function)
#   DeviceInstanceUpper - Device Instance Upper (ISO Function Instance)
#   DeviceInstanceLower - Device Instance Lower (ISO ECU Instance)
#
# OUTPUTS:
#   a pointer to a Python integer object, TRUE if the initialization was
#   successful, FALSE (zero) otherwise.
#
# NOTES:
#
# VERSION HISTORY:
#
# Version: 1.00   Date: 2006-09-19   By: Tom Lightfoot
#   - Added header
#
#####

class clBusConnection:
    def __init__(self,
                  devfunc=XB_eFUNC_CODE_DIAGNOSTIC_TERMINAL,
                  devinstupper=0,
                  devinstlower=0):
        # Initialize the database

```

```

self.db = PgDesc.clXbDatabase()

# Keep track of my device function so that we know how many
# requested configuration messages to expect
self.devfunc = devfunc

# Initialize the low level XanBus interface
if not _xanbus.Init(devinstlower, devinstupper, devfunc):
    raise XbInitError("Initialization error")

# Shouldn't have any devices in the list at this stage
if self.fnDevices():
    raise XbInitError("Devices left over from last connection")

# Check versions
self.version = _xanbus.GetVersion()
if self.db.version != self.version:
    print "Database version: ", self.db.version
    print "XanBus DLL version: ", self.version
    raise XbInitError("Database and DLL version mismatch")

# Make sure that we can receive all status messages sent
for pgn in self.db.sts_pgns:
    self.fnListen(pgn)

# List of receivers
self.devhandlers = {}
self.nextid = 0
self.evthandlers = []

# Set the receiver function
_xanbus.SetReceiveCB(self._fnReceiveCb)

# Set the other event handling callbacks
self.events = []
self.eventlock = threading.Lock()
_xanbus.SetNodeChangeCB(self._fnNodeChangeCb)
_xanbus.SetNetIndicCB(self._fnNetIndicCb)
_xanbus.SetErrorCB(self._fnErrorCb)

# Start the tick thread
self.closed = False
self.online = False
self.ticker = threading.Timer(XB_TICK_PERIOD, self._fnTick)
self.ticker.start()

def _fnTick(self):
    _xanbus.Tick()
    if not self.online:
        if _xanbus.OnLine():
            self.online = True

    if not self.closed:
        self.ticker = threading.Timer(XB_TICK_PERIOD, self._fnTick)
        self.ticker.start()

def fnClose(self):
    # Make sure all connections are closed
    for conlist in self.devhandlers.values():
        if conlist:
            raise PyXanBusError("Can't close with device connections still open")

    self.ticker.cancel()
    self.closed = True

# Wait for current tick cycle to finish before turning off the
# low level interface
time.sleep(2 * POLL_PERIOD)
_xanbus.Close()

# Disable callbacks
_xanbus.ClrReceiveCB()
_xanbus.ClrNodeChangeCB()
_xanbus.ClrNetIndicCB()
_xanbus.ClrErrorCB()

```

```

# Get the bus state
def fnGetState(self):
    return fnBusState(_xanbus.GetBusState())

# Get my address on the bus
def fnMyAddr(self):
    return _xanbus.GetMyAddr()

# Return a list of devices on the network, including NAME and address
def fnDevices(self):
    devlist = []
    for i in range(XB_MAX_NODES):
        devtup = _xanbus.FindDevice(i)
        if devtup:
            addr, msg = devtup
            # Each tuple in the returned list consists of the
            # device address and a dictionary of the interpreted
            # name components
            namedict = self.db.fnInterpret(XB_PGN_ISO_ADDR_CLAIM, None, msg, 0, None)
            namemsg = clMessage(namedict, self.db)
            devlist.append((addr, namemsg))

    return devlist

# Request a PGN from a device
def fnRequest(self, pgn, dev):
    # Allow receipt of the message and then request it
    _xanbus.CtrlDynamic(pgn, True)
    _xanbus.ISOReq(pgn, dev)

# Listen for a PGN on the network
def fnListen(self, pgn):
    _xanbus.CtrlDynamic(pgn, True)

# Send a PGN with group function specified
def fnSendEx(self, pgn, grpfunc, addr, msgbuf, extrasbuf=None):
    if not _xanbus.SendEx(pgn, grpfunc, addr, msgbuf, extrasbuf):
        raise PyXanBusError("Send error: %d" % self.fnGetErrorNum())

# Send a PGN on the bus
def fnSend(self, pgn, addr, msgbuf, extrasbuf=None):
    self.fnSendEx(pgn, PGN_GRP_FUNC_DO_NOT_CARE, addr, msgbuf, extrasbuf)

# XanBus stack error callback, called from the Tick thread
def _fnErrorCb(self, errno, tagstr):
    if self.closed:
        return

    self.eventlock.acquire()
    self.events.append(("error", (errno, tagstr)))
    self.eventlock.release()
    self.fnUpdateEventMonitors()
    if tagstr:
        print "%s: Error number %d" % (tagstr, errno)
    else:
        print "XanBus error %d" % errno

# Network state indication callback, called from the Tick thread
def _fnNetIndicCb(self, state):
    try:
        if self.closed:
            return

        self.eventlock.acquire()
        self.events.append(("indic", fnBusState(state)))
        self.eventlock.release()
        self.fnUpdateEventMonitors()
        print "Bus state is %s" % fnBusState(state)

    except:
        print "Exception in _fnNetIndicCb"

# Node change callback, called from the Tick thread
def _fnNodeChangeCb(self, changenum, oldaddr, newaddr, name):
    try:

```

```

    if self.closed:
        return

    self.eventlock.acquire()
    if changenum == XB_eNODE_CHANGE_NEW_NODE:
        print "New node %d" % newaddr
        if name:
            addrclaim = clMessage(self.db.fnInterpret(XB_PGN_ISO_ADDR_CLAIM, None, name, 0, None),
                                  self.db)
            self.events.append(('new', (newaddr, addrclaim)))
            self.fnUpdateEventMonitors()
        else:
            print "No name message passed"

    elif changenum == XB_eNODE_CHANGE_ADDRESS:
        print "Node %d changed address to %d" % (oldaddr, newaddr)
        self.events.append(('change', (oldaddr, newaddr)))
        self.fnUpdateEventMonitors()

    elif changenum == XB_eNODE_CHANGE_BUMPED_OFF:
        print "Node %d bumped off" % oldaddr
        self.events.append(('bumped', oldaddr))
        self.fnUpdateEventMonitors()

    elif changenum == XB_eNODE_CHANGE_TIMED_OUT:
        print "Node %d timed out" % oldaddr
        self.events.append(('timeout', oldaddr))
        self.fnUpdateEventMonitors()

    else:
        print "Unknown node change %d" % changenum

    self.eventlock.release()

except:
    print "Exception in _fnNodeChangeCb"

# Get an event from the event queue if there is one
def fnGetEvent(self):
    self.eventlock.acquire()
    if self.events:
        event = self.events[0]
        del(self.events[0])

    else:
        event = None

    self.eventlock.release()

    return event

# Get the latest error number
def fnGetErrorNum(self):
    return _xanbus.GetErrorNum()

# Receive callback, called from the Tick thread
def _fnReceiveCb(self, pgn, grpfunc, src, dest, msg, numextras=0, extras=None):
    if self.devhandlers.has_key(src):
        msgdict = self.db.fnInterpret(pgn, grpfunc, msg, numextras, extras)
        msgobj = clMessage(msgdict, self.db)

        for devcon in self.devhandlers[src].values():
            devcon.fnReceive(msgobj, src, dest)

# Register a device connection
def fnRegisterDevConnection(self, devcon, devaddr):
    if not self.devhandlers.has_key(devaddr):
        self.devhandlers[devaddr] = {}

    self.devhandlers[devaddr][devcon.conid] = devcon

# Unregister a device connection
def fnUnregisterDevConnection(self, devaddr, conid):
    if self.devhandlers.has_key(devaddr):
        if self.devhandlers[devaddr].has_key(conid):
            del(self.devhandlers[devaddr][conid])

```

```

# Get a connection to one of the devices
def fnGetConnection(self, devaddr):
    return clDeviceConnection(self, devaddr, self.nextid)
    self.nextid += 1

# Register a connection by an event monitor
def fnRegisterEventMonitor(self, monitor):
    if monitor not in self.evthandlers:
        self.evthandlers.append(monitor)

def fnUnregisterEventMonitor(self, monitor):
    for i in range(len(self.evthandlers)):
        if self.evthandlers[i] == monitor:
            del(self.evthandlers[i])
            break

# Send the most recent event to each event monitor
def fnUpdateEventMonitors(self):
    event = self.events[-1]
    for monitor in self.evthandlers:
        monitor.fnAddEvent(event)

# Get an event monitor to the bus
def fnGetEventMonitor(self):
    return clEventMonitor(self)

# Turn on trace
def fnTraceOn(self):
    _xanbus.TraceOn()

# Turn off trace
def fnTraceOff(self):
    _xanbus.TraceOff()

```

```

#####
#
# CLASS NAME: clDeviceConnection
#
# PURPOSE:
#     Connection object for a device
#
# INPUTS:
#     bus - The bus connection object to connect to
#     dev - The CAN address of the device to connect to
#
# OUTPUTS:
#
# NOTES:
#     One would not normally create an object with this class directly.
#     use the bus connection object's fnGetConnection() method
#     instead.
#
# VERSION HISTORY:
#
# Version: 1.00 Date: 2006-09-19 By: Tom Lightfoot
#     - Added header
#
#####

```

```

class clDeviceConnection:
    def __init__(self, bus, dev, conid):
        self.bus = bus
        self.dev = dev
        self.conid = conid
        bus.fnRegisterDevConnection(self, dev)
        self.replyqueue = {}
        self.expected_pgn = 0
        self.queueLock = threading.Lock()
        self.broadcasts = {}
        self.bcstlock = threading.Lock()

        # Initialize the timing statistics
        self.bcst_time = {}
        self.bcst_period = {}

```

```

def fnClose(self):
    self.bus.fnUnregisterDevConnection(self.dev, self.conid)

# Just send an ISO request for a PGN. Other processes will get the replies
def fnSendRequest(self, pgn):
    self.expected_pgn = pgn
    self.bus.fnRequest(pgn, self.dev)

def fnSendRequestByMnem(self, mnem):
    pgn = self.bus.db.fnFindPgn(mnem)
    self.expected_pgn = pgn
    self.bus.fnRequest(pgn, self.dev)

# Request a pgn from the connection and return a list of the reply messages
def fnRequest(self, pgn):
    # Send the request
    self.bus.fnRequest(pgn, self.dev)

    # Initialize list of reply messages
    replies = []
    self.expected_pgn = pgn

    # Several messages may be in the reply so keep receiving until
    # the other device stops sending
    poll = 0
    while True:
        reply = self.fnPollReply()
        if reply:
            pgn_mnem = reply.fnGetMnem()
            if pgn_mnem == 'ISOack':
                break

            # Is this the message we're expecting?
            if reply.fnGetPgn() == self.expected_pgn:
                replies.append(reply)

                # Configuration PGNs sent to a diagnostic terminal have
                # 3 times the message count
                msg_count = reply.fnGetRawValue('MsgCount')
                if msg_count:
                    if ((self.bus.devfunc == XB_eFUNC_CODE_DIAGNOSTIC_TERMINAL)
                        and (pgn_mnem.find('Cfg') >= 0)
                        and reply.fnGetValue('CfgType')):
                        msg_count *= 3

                # Have we received all of the replies that we are expecting?
                if len(replies) == msg_count:
                    break

            # If this an ISO Ack then we're done
            elif pgn_mnem == 'ISOack':
                replies.append(reply)
                break

        else:
            # Ignore unexpected message
            continue

    else:
        poll += 1
        if poll >= MAX_POLLS:
            break

    time.sleep(POLL_PERIOD)

    # Return the list of messages
    return replies

# Request a PGN by mnemonic
def fnRequestByMnem(self, mnem):
    pgn = self.bus.db.fnFindPgn(mnem)
    return self.fnRequest(pgn)

# Get a message from the reply queue
def fnGetReply(self):

```

```

for poll in range(MAX_POLLS):
    self.queue.lock.acquire()
    if len(self.replyqueue) == 0:
        # No reply, unlock the queue and wait some more
        self.queue.lock.release()
        time.sleep(POLL_PERIOD)

    else:
        # Reply in queue, remove and return
        replymsg = self.replyqueue[0]
        del(self.replyqueue[0])
        self.queue.lock.release()
        return replymsg

# Timeout
return None

# Poll the reply queue for a message (no waiting, no timeouts)
def fnPollReply(self):
    self.queue.lock.acquire()
    if len(self.replyqueue) == 0:
        # No reply, unlock the queue and wait some more
        self.queue.lock.release()
        return None

    else:
        # Reply in queue, remove and return
        replymsg = self.replyqueue[0]
        del(self.replyqueue[0])
        self.queue.lock.release()
        return replymsg

def fnReceive(self, msgobj, src, dest):
    # Is this a reply directly to me?
    pgn = msgobj.fnGetPgn()
    if ((dest == self.bus.fnMyAddr())
        or (pgn == XB_PGN_ISO_ACK)
        or ((pgn == self.expected_pgn)
            and msgobj.fnGetMsgCount() > 0)):
        # Add this message to the reply queue
        self.queue.lock.acquire()
        self.replyqueue.append(msgobj)
        self.queue.lock.release()

    else:
        # Broadcast

        # What time is it now?
        now = time.clock()

        # Look for an id type field
        idvalue = msgobj.fnGetIdVal()

        # Lock out the main thread before saving the message
        self.bcstlock.acquire()
        key = (pgn, idvalue)
        self.broadcasts[key] = msgobj

        # Keep the timing statistics
        if self.bcst_time.has_key(key):
            self.bcst_period[key] = now - self.bcst_time[key]
            self.bcst_time[key] = now

        self.bcstlock.release()

# Get the list of broadcast pgns received
def fnGetBcstPgns(self):
    self.bcstlock.acquire()
    pgnlist = self.broadcasts.keys()
    self.bcstlock.release()
    return pgnlist

# Get the content of the latest specified broadcast PGN
def fnGetBcst(self, pgn, idval=None):
    bcstkey = (pgn, idval)
    self.bcstlock.acquire()

```

```

        if self.broadcasts.has_key(bcstkey):
            retval = self.broadcasts[bcstkey]
        else:
            retval = None

        self.bcstlock.release()

    return retval

# Bring out the broadcast statistics
def fnGetBcstPeriod(self, pgn, idval=None):
    bcstkey = (pgn, idval)
    self.bcstlock.acquire()
    if self.bcst_period.has_key(bcstkey):
        period = self.bcst_period[bcstkey]
    else:
        period = None

    self.bcstlock.release()

    return period

# Send a message
def fnSend(self, msg):
    msgbuf, extrasbuf = self.bus.db.fnPack(msg.msgdict)
    self.bus.fnSend(msg.fnGetPgn(),
                    self.dev,
                    msgbuf,
                    extrasbuf)

# Wait for an ack
def fnGetAck(self, req):
    ackmsg = self.fnGetReply()
    if ackmsg is None:
        raise XbAckError("No reply", req)

    if ackmsg.fnGetMnem() != 'ISOAck':
        raise XbAckError("Unexpected reply (%s)" % ackmsg.fnGetMnem(), req)

    ctrlbyte = ackmsg.fnGetValue('CtrlByte')
    if ctrlbyte != 'Ack':
        reason = ackmsg.fnGetValue('GroupFunctionValue')
        raise XbAckError("%s: %s" % (ctrlbyte, reason), req)

# Make a message object for a given PGN mnemonic
def fnMakeMsg(self, pgn_mnem, ntuples=0):
    msgdict = self.bus.db.fnMakeNew(pgn_mnem, ntuples)
    return clMessage(msgdict, self.bus.db)

# Send a complex request for a string configuration PGN
def fnReqStringCfg(self, stringid):
    # Make sure receipt is enabled for StringCfg
    StringCfg = self.bus.db.fnFindPgn('StringCfg')

    # Create a complex request
    msg = self.fnMakeMsg('ComplexReq', ntuples=1)
    msg.fnSetRawValue('ParamGrpNum', StringCfg)
    msg.fnSetRawValue('Field', 4, 0)
    msg.fnSetRawValue('FieldValue', stringid, 0)

    # Let the receiver know we're expecting StringCfg as a reply
    self.expected_pgn = StringCfg

    # Send the request
    self.fnSend(msg)

# Get a string according to its ID
def fnGetString(self, stringid):
    # Request string config by the ID
    self.fnReqStringCfg(stringid)

    # Several messages may be in the reply so keep receiving until
    # the other device stops sending
    for poll in range(MAX_POLLS):
        reply = self.fnPollReply()
        if reply:

```

```

        pgn_mnem = reply.fnGetMnem()
        if pgn_mnem == 'ISOAck':
            reply = None
            break

        # Is this the message we're expecting?
        if pgn_mnem == 'StringCfg':
            if reply.fnGetValue('CfgType') == 'User':
                break

    else:
        time.sleep(POLL_PERIOD)

    if reply is None:
        return None

    else:
        string = ''
        for i in range(reply.fnGetNumTuples()):
            byteval = reply.fnGetRawValue('Byte', i)
            if byteval == 0:
                break
            else:
                string += chr(byteval)

        return string

# For diagnostic terminals only, get all string Cfgs according
# to StringId
def fnGetStrings(self, stringid):
    # Request string config by the ID
    self.fnReqStringCfg(stringid)

    # Create a list of CfgType/String tuples
    stringlist = []

    # Get the responses
    while True:
        reply = self.fnGetReply()
        if reply is None:
            break

        elif reply.fnGetMnem() != 'StringCfg':
            break

        else:
            # Get the config type and string from the StringCfg
            cfg_type = reply.fnGetValue('CfgType')
            string = ''
            for i in range(reply.fnGetNumTuples()):
                byteval = reply.fnGetRawValue('Byte', i)
                if byteval == 0:
                    break
                else:
                    string += chr(byteval)

            stringlist.append((cfg_type, string))

    return stringlist

# Change a string config
def fnSetString(self, stringid, newstring):
    # Request the current configuration for this id
    request = self.fnMakeMsg('ComplexReq')
    request.fnSetRawValue('ParamGrpNum', self.bus.db.fnFindPgn('StringCfg'))
    request.fnSetRawValue('Field', 4)
    request.fnSetRawValue('FieldValue', stringid)
    self.fnSend(request)

    stringcfg = None
    while True:
        reply = self.fnGetReply()
        if reply is None:
            break

        elif reply.fnGetMnem() != 'StringCfg':

```

```

        break

    else:
        if reply.fnGetValue('CfgType') == 'User':
            stringcfg = reply

    if not stringcfg:
        raise PyXanBusError("No StringCfg received")

    # Turn reported config into a commanded config
    stringcfg.fnSetRawValue("MsgCount", 0)

    # Delete the old string
    stringcfg.fnClearTuples()

    # Write the new string
    for i in range(len(newstring)):
        stringcfg.fnAddTuple([{'mnem' : 'Byte',
                               'rawvalue' : ord(newstring[i]),
                               'value' : newstring[i]}])

    stringcfg.fnAddTuple([{'mnem' : 'Byte',
                           'rawvalue' : 0,
                           'value' : "NULL"}])

    # Send new name to device
    self.fnSend(stringcfg)

    # Hope for ACK
    reply = self.fnGetReply()
    if reply:
        mnem = reply.fnGetMnem()
        if mnem == 'ISOack':
            ctrlbyte = reply.fnGetValue('CtrlByte')
            if ctrlbyte != 'Ack':
                reason = reply.fnGetValue("GroupFunctionValue")
                raise PyXanBusError("Device replied with %s (%s) to new StrinCfg" % (ctrlbyte, reason))
        else:
            raise PyXanBusError("Unexpected pgn: %s" % mnem)

    else:
        raise PyXanBusError("No reply to new StringCfg")

# Get the device's address claim message object
def fnGetNAME(self):
    rawclaim = _xanbus.GetNAME(self.dev)
    claimdict = self.bus.db.fnInterpret(XB_PGN_ISO_ADDR_CLAIM,
                                         None,
                                         rawclaim,
                                         0, None)

    return clMessage(claimdict, self.bus.db)

```

```

#####
#
# CLASS NAME: clMessage
#
# PURPOSE:
#   Interpreted XanBus message, providing a number of convenient methods
#   for getting and manipulating the data by signal and message mnemonics
#
# INPUTS:
#   msgdict - the complex dictionary structure provided by the
#             PGN Descriptor's fnInterpret method
#
# OUTPUTS:
#
# NOTES:
#
# VERSION HISTORY:
#
# Version: 1.00 Date: 2006-09-19 By: Tom Lightfoot
#   - Added header
#
#####

```

```

class clMessage:
    def __init__(self, msgdict, db):
        self.msgdict = msgdict
        self.db = db

    # Make a nice printout of the message's contents
    def fnPrettyPrint(self):
        print self

    # Print all of the signals of a message
    def fnPrintSignals(self):
        print self.fnSignalsToText()

    # Create a text representation of a message
    def __str__(self):
        msgtext = "%s (%06x):\n" % (self.msgdict['mnem'], self.msgdict['pgn'])
        msgtext += self.fnSignalsToText()
        return msgtext

    # Turn all of the signals into a text string
    def fnSignalsToText(self):
        sigtext = ''

        # Fixed length portion
        for sigdict in self.msgdict['fixedsignals']:
            sigtext += self.fnSignalToText(sigdict)

        # Tuples of variable length portion (if any)
        tupnum = 1
        for tup in self.msgdict['tuples']:
            sigtext += "Tuple %d\n" % tupnum
            tupnum += 1
            for sigdict in tup:
                sigtext += self.fnSignalToText(sigdict)

        return sigtext

    # Create a text representation of a signal
    def fnSignalToText(self, sigdict):
        return "  %-20s : %s (%s)\n" % (sigdict['mnem'], sigdict['value'], sigdict['rawvalue'])

    # Get the number of tuples in a message
    def fnGetNumTuples(self):
        if self.msgdict.has_key('tuples'):
            return len(self.msgdict['tuples'])
        else:
            return 0

    # Get an attribute of a selected signal
    def fnGetSigAttr(self, attrname, sigmnem, tupindex=None):
        # Are we looking for a field in the fixed-length portion?
        if tupindex is None:
            siglist = self.msgdict['fixedsignals']
        else:
            siglist = self.msgdict['tuples'][tupindex]

        # Scan through the appropriate field list
        for sigdict in siglist:
            if sigdict['mnem'] == sigmnem:
                return sigdict[attrname]

        # Named field not found
        return None

    # Get the value of a named field
    def fnGetValue(self, sigmnem, tupindex=None):
        return self.fnGetSigAttr('value', sigmnem, tupindex)

    # Get the raw value of a named field
    def fnGetRawValue(self, sigmnem, tupindex=None):
        return self.fnGetSigAttr('rawvalue', sigmnem, tupindex)

    # Set the raw value of a named field
    def fnSetRawValue(self, sigmnem, val, tupindex=None):
        if tupindex is None:

```

```

        siglist = self.msgdict['fixedsignals']
    else:
        siglist = self.msgdict['tuples'][tupindex]

    for sigdict in siglist:
        if sigdict['mnem'] == sigmnem:
            sigdict['rawvalue'] = val
            return

    raise XbMsgError("fnSetRawValue: Could not find signal %s" % sigmnem)

# Set a named field, converting the interpreted value back to its raw equivalent
def fnSetValue(self, sigmnem, val, tupindex=None):
    if tupindex is None:
        siglist = self.msgdict['fixedsignals']
    else:
        siglist = self.msgdict['tuples'][tupindex]

    for sigdict in siglist:
        if sigdict['mnem'] == sigmnem:
            sigdict['value'] = val

            # Convert the value into the raw integer
            self.db.fnBackConvertSignal(sigdict)
            return

    raise XbMsgError("fnSetValue: Could not find signal %s" % sigmnem)

# Get the variable length string at the end of a StringCfg message
def fnGetString(self):
    if self.fnGetMnem() != "StringCfg":
        return ''

    # Make a string from the tuple values, stopping at the null
    string = ''
    for i in range(self.fnGetNumTuples()):
        byteval = self.fnGetRawValue('Byte', i)
        if byteval == 0:
            break
        else:
            string += chr(byteval)

    return string

# Get the message count field
def fnGetMsgCount(self):
    val = self.fnGetValue('MsgCount')
    if val is None:
        return 0
    else:
        return val

# Get the mnemonic of a message's ID field
def fnGetIdMnem(self):
    # Scan through the fixed signals, looking for one who's
    # mnemonic ends in 'Id'
    for sigdict in self.msgdict['fixedsignals']:
        if sigdict['mnem'].endswith('Id'):
            return sigdict['mnem']

    return None

# Get the id field of a message
def fnGetIdVal(self):
    # Scan through the fixed signals, looking for one who's
    # mnemonic ends in 'Id'
    for sigdict in self.msgdict['fixedsignals']:
        if sigdict['mnem'].endswith('Id'):
            return sigdict['value']

    # ID field not found
    return None

# Get the raw ID field of a message
def fnGetIdRawVal(self):
    # Scan through the fixed signals, looking for one who's

```

```

# mnemonic ends in 'Id'
for sigdict in self.msgdict['fixedsignals']:
    if sigdict['mnem'].endswith('Id'):
        return sigdict['rawvalue']

# ID field not found
return None

# Find the configuration limits from a message
def fnGetMinMax(self, sig_mnem, tupindex=None):
    # Default no-find values
    rawmin = None
    rawmax = None

    # Signal mnemonics are same as base, but with Min or Max
    minsig = sig_mnem + 'Min'
    maxsig = sig_mnem + 'Max'

    # First try the limits for the tuple
    if tupindex is not None:
        tup = self.msgdict['tuples'][tupindex]
        for sigdict in tup:
            if sigdict['mnem'] == minsig:
                rawmin = sigdict['rawvalue']

            if sigdict['mnem'] == maxsig:
                rawmax = sigdict['rawvalue']

    # Try the main body if not found in the tuple
    if rawmax is None:
        for sigdict in self.msgdict['fixedsignals']:
            if sigdict['mnem'] == minsig:
                rawmin = sigdict['rawvalue']

            if sigdict['mnem'] == maxsig:
                rawmax = sigdict['rawvalue']

    # Return the found limits or None
    return (rawmin, rawmax)

# Get the PGN mnemonic of a message
def fnGetMnem(self):
    return self.msgdict['mnem']

# Get the PGN of a message
def fnGetPgn(self):
    return self.msgdict['pgn']

# Get a list of the fixed signal mnemonics
def fnGetSignalMnems(self, tupindex=None):
    siglist = []

    # Get names from fixed length portion of message
    if tupindex is None:
        msg_segment = self.msgdict['fixedsignals']

    elif self.fnGetNumTuples() > tupindex:
        msg_segment = self.msgdict['tuples'][tupindex]

    else:
        # Tuple specified but this message has no tuples
        return []

    # Get the list of signals from the desired message segment
    for sigdict in msg_segment:
        siglist.append(sigdict['mnem'])

    return siglist

# Convert a message to an XML string
def fnToXml(self):
    # XbMsg element with attributes
    xml = '<XbMsg'
    for attr in ['pgn', 'mnem']:
        xml += ' %s="%s"' % (attr, self.msgdict[attr])

```

```

xml += '>\n'

# Fixed signals
xml += '<fixedsignals>\n'
for sigdict in self.msgdict['fixedsignals']:
    xml += self._fnSigToXml(sigdict)
xml += '</fixedsignals>\n'

# Variable number of tuples element
xml += '<tuples>\n'
for siglist in self.msgdict['tuples']:
    xml += '<tuple>\n'
    for sigdict in siglist:
        xml += self._fnSigToXml(sigdict)
    xml += '</tuple>\n'
xml += '</tuples>\n'

# Closing
xml += '</XbMsg>\n'

return xml

# Convert a message signal into an XML string
def _fnSigToXml(self, sigdict):
    xml = "<signal"
    for key in sigdict.keys():
        # Remove nulls from strings
        value = sigdict[key]
        if type(value) == type('string'):
            value = value.replace('\0', '')

        xml += ' %s="%s"' % (key, value)
    xml += ">\n"
    return xml

# Convert a message into an XML Document Object
def fnToDom(self, dom):
    # Create the empty element
    msgdom = dom.createElement('XbMsg')

    # Add identifier attributes
    for attr in ['pgn', 'mnem']:
        msgdom.setAttribute(attr, str(self.msgdict[attr]))

    # Add the fixed signals
    fixeddom = dom.createElement('fixedsignals')
    for sigdict in self.msgdict['fixedsignals']:
        sigdom = self.fnToSigDom(dom, sigdict)
        fixeddom.appendChild(sigdom)

    msgdom.appendChild(fixeddom)

    # Add the tuples
    tuplesdom = dom.createElement('tuples')
    for siglist in self.msgdict['tuples']:
        tupledom = dom.createElement('tuple')
        for sigdict in siglist:
            sigdom = self.fnToSigDom(dom, sigdict)
            tupledom.appendChild(sigdom)

        tuplesdom.appendChild(tupledom)

    msgdom.appendChild(tuplesdom)

    return msgdom

# Convert a field of a message into an XML Document Object
def fnToSigDom(self, dom, sigdict):
    # Create the empty signal element
    sigdom = dom.createElement('signal')

    # All key values in the dictionary are added in as attributes
    for key in sigdict.keys():
        # Get rid of funky characters that will break the XML
        value = str(sigdict[key])

```

```

        value = re.sub('[^_ /().?@0-9a-zA-Z]', '', value)
        sigdom.setAttribute(key, value)

    return sigdom

# Clear out all of a variable length PGNs tuples
def fnClearTuples(self):
    del(self.msgdict['tuples'][:])

# Add a tuple to a variable length PGN
def fnAddTuple(self, siglist):
    self.msgdict['tuples'].append(siglist)

# Monitor for XanBus events
class clEventManager:
    def __init__(self, xb):
        self.xb = xb
        self.eventqueue = []
        self.queueLock = threading.Lock()

        xb.fnRegisterEventManager(self)

    # Method for bus connection to call to add an event
    def fnAddEvent(self, event):
        self.queueLock.acquire()
        self.eventqueue.append(event)
        self.queueLock.release()

    # Method for the application to get the next event, if any
    def fnGetEvent(self):
        self.queueLock.acquire()
        if self.eventqueue:
            event = self.eventqueue[0]
            del(self.eventqueue[0])

        else:
            event = None
            self.queueLock.release()

        return event

    # Method for the application to close the monitor
    def fnClose(self):
        self.xb.fnUnregisterEventManager(self)

# Exception classes
class PyXanBusError(Exception):
    pass

class XbInitError(PyXanBusError):
    pass

class XbMsgError(PyXanBusError):
    pass

class XbAckError(PyXanBusError):
    def __init__(self, errstr, reqmsg):
        # Keep track of the error message
        self.errstr = errstr

        # And the XanBus message that triggered the Nak
        self.reqmsg = reqmsg

#####
#                                     Function Definitions
#####

# Interpret the bus state number
def fnBusState(statenum):
    state = "unknown"
    if statenum == XB_eNETWORK_STATE_ERROR_ACTIVE:
        state = "active"
    elif statenum == XB_eNETWORK_STATE_ERROR_PASSIVE:
        state = "passive"

```

```

elif XB_eNETWORK_STATE_BUS_OFF:
    state = "bus off"

return state

# Module test function
def fnTest():
    xb = clBusConnection()
    print "XanBus Version %s" % xb.version
    print "My address is %d" % xb.fnMyAddr()
    print "Bus state: %s" % xb.fnGetState()

    # Wait a while for other devices to appear
    time.sleep(2)

    # See who is on the list
    devlist = xb.fnDevices()
    print "%d devices on network" % len(devlist)
    if devlist:
        for devaddr, addrclaim in devlist:
            print "Device at address %d:" % devaddr
            addrclaim.fnPrintSignals()

        dev = devlist[0]
        addr = dev[0]
        devcon = xb.fnGetConnection(addr)

        # Request a pgn
        xb.fnTraceOn()
        reqpgn = 0x012500
        for i in range(1, NUM_TESTS):
            print "%d Requesting pgn %s" % (i, xb.db.fnPgnMnem(reqpgn))
            replies = devcon.fnRequest(reqpgn)
            print "%d replies received" % len(replies)
            for reply in replies:
                reply.fnPrettyPrint()

        xb.fnTraceOff()

        # Request product info status
        reqpgn = xb.db.fnFindPgn("ProdInfoSts")
        print "Requesting pgn %s" % xb.db.fnPgnMnem(reqpgn)
        replies = devcon.fnRequest(reqpgn)
        print "%d replies received" % len(replies)
        for reply in replies:
            reply.fnPrettyPrint()

        # Request software version status
        reqpgn = xb.db.fnFindPgn("SwVerSts")
        print "Requesting pgn %s" % xb.db.fnPgnMnem(reqpgn)
        replies = devcon.fnRequest(reqpgn)
        print "%d replies received" % len(replies)
        for reply in replies:
            reply.fnPrettyPrint()

        # Try requesting AssocSts
        reqpgn = xb.db.fnFindPgn("AssocSts")
        print "Requesting AssocSts"
        replies = devcon.fnRequest(reqpgn)
        print "%d replies received" % len(replies)
        for reply in replies:
            reply.fnPrettyPrint()

        # Request supported pgn lists
        reqpgn = 0x01ee00
        print "Requesting PGN list"
        replies = devcon.fnRequest(reqpgn)
        print "%d replies received" % len(replies)
        for reply in replies:
            reply.fnPrettyPrint()

        # Try a complex request to get a string
        print "Complex request for StringCfg(0)"
        string = devcon.fnGetString(0)
        if string is None:
            print "No string received"

```

```
else:
    print string

# See what the broadcasts are
time.sleep(1)
broadcasts = devcon.fnGetBcstPgns()
print "%d Broadcast PGNs:" % len(broadcasts)
for pgn, idval in broadcasts:
    msg = devcon.fnGetBcst(pgn, idval)
    msg.fnPrettyPrint()

print "New ActiveFaultSts2 with 3 tuples:"
print str(devcon.fnMakeMsg("ActiveFaultSts2", 3))

# Close the connection to the device
devcon.fnClose()

# Report state when closing
time.sleep(2)
print "My address is %d" % xb.fnMyAddr()
print "Bus state: %s" % xb.fnGetState()

xb.fnClose()

if __name__ == "__main__":
    fnTest()
```

FullPgnList

This is a full PGN list taken from the Xanbus database: Version 1.4.0 Build 726

<u>PGN Mnemonic</u>	<u>PGN</u>	<u>PG Name</u>
<u>AcInCfg</u>	0x12600 (ID: 2749)	AC Input Configuration
<u>AcInCfgOf</u>	0x14E00 (ID: -48150255)	AC Input Configuration - Over Frequency
<u>AcInCfgOvAvg</u>	0x10C00 (ID: 2723)	AC Input Configuration - Over Voltage - Average
<u>AcInCfgOvPeak</u>	0x10B00 (ID: 2722)	AC Input Configuration - Over Voltage - Peak
<u>AcInCfgOvRms</u>	0x10A00 (ID: 2721)	AC Input Configuration - Over Voltage - RMS
<u>AcInCfgUf</u>	0x14F00 (ID: 364846590)	AC Input Configuration - Under Frequency
<u>AcInCfgUvAvg</u>	0x10900 (ID: 2720)	AC Input Configuration - Under Voltage - Average
<u>AcInCfgUvPeak</u>	0x10800 (ID: 2719)	AC Input Configuration - Under Voltage - Peak
<u>AcInCfgUvRms</u>	0x10700 (ID: 2718)	AC Input Configuration - Under Voltage - RMS
<u>AcInStsAvg</u>	0x1F005 (ID: 2693)	AC Input Status - Average
<u>AcInStsPeak</u>	0x1F004 (ID: 2692)	AC Input Status - Peak
<u>AcInStsRms</u>	0x1F003 (ID: 2691)	AC Input Status - RMS
<u>AcOutCfg</u>	0x10600 (ID: 2717)	AC Output Configuration
<u>AcOutCfgOvRms</u>	0x15100 (ID: 1074878349)	AC Output Configuration - Over Voltage - RMS
<u>AcOutCfgUvRms</u>	0x15200 (ID: -1175121462)	AC Output Configuration - Under Voltage - RMS
<u>AcOutStsAvg</u>	0x1F008 (ID: 2696)	AC Output Status - Average
<u>AcOutStsPeak</u>	0x1F007 (ID: 2695)	AC Output Status - Peak
<u>AcOutStsRms</u>	0x1F006 (ID: 2694)	AC Output Status - RMS
<u>ActiveFaultSts</u>	0x1F80B (ID: 2706)	Active Fault Status
<u>ActiveFaultSts2</u>	0x1F8C6 (ID: -140626159)	Active Fault Status (Deux)
<u>ActiveWarnSts</u>	0x1F80D (ID: 1055545630)	Active Warning Status
<u>ActiveWarnSts2</u>	0x1F8C5 (ID: 1140723024)	Active Warning Status (Deux)
<u>AcXferSwCfg</u>	0x1C400 (ID: 1140795486)	AC Transfer Switch Configuration
<u>AcXferSwSts</u>	0x1F0BF (ID: -207713145)	AC Transfer Switch Status
<u>AgsCfg</u>	0x12A00 (ID: 2753)	AGS Configuration
<u>AgsCfgAcI</u>	0x13E00 (ID: -1627600155)	AGS Configuration - AC Current
<u>AgsCfgAcPwr</u>	0x15700 (ID: -1027437734)	AGS Configuration - AC Power
<u>AgsCfgBattSoc</u>	0x13D00 (ID: -1146091276)	AGS Configuration - Battery SOC
<u>AgsCfgCntctCls</u>	0x13F00 (ID: 284780610)	AGS Configuration - Contact Closure
<u>AgsCfgDcI</u>	0x15600 (ID: 1084473693)	AGS Configuration - DC Current
<u>AgsCfgDcV</u>	0x13C00 (ID: 1043087263)	AGS Configuration - DC Voltage
<u>AgsCfgGen</u>	0x12B00 (ID: 2754)	AGS Configuration - Generator
<u>AgsCtrlCmd</u>	0x11F00 (ID: 1069285978)	AGS Control Command
<u>AgsStatsGen</u>	0x1F018 (ID: 1046214140)	AGS Statistics - Generator
<u>AgsSts</u>	0x1F011 (ID: 2709)	AGS Status

<u>AgsStsGen</u>	0x1F017 (ID: 2710)	AGS Status - Generator
<u>ApsCfgOv</u>	0x10E00 (ID: 2725)	Auxiliary Power Supply Configuration - Over Voltage
<u>ApsCfgUv</u>	0x10D00 (ID: 2724)	Auxiliary Power Supply Configuration - Under Voltage
<u>ApsSts</u>	0x1F009 (ID: 2697)	Auxiliary Power Supply Status
<u>AssocCfg</u>	0x12500 (ID: 2748)	Association Configuration
<u>AssocSts</u>	0x13600 (ID: 2765)	Association Status
<u>BattMonCfg</u>	0x12900 (ID: 2752)	Battery Monitor Configuration
<u>BattMonStatsBatt</u>	0x1F015 (ID: 2708)	Battery Monitor Statistics - Battery
<u>BattMonSts</u>	0x1F01B (ID: -282122421)	Battery Monitor Status
<u>BattSts2</u>	0x1F0C4 (ID: -1490038293)	Battery Status (Deux)
<u>CalibCmd</u>	0x11E00 (ID: 1066253150)	Calibration Command
<u>CalibDataSts</u>	0x1F026 (ID: 1099419207)	Calibration Data Status
<u>Cfg</u>	0x14700 (ID: 1800987186)	Configuration
<u>ChgCfg</u>	0x12C00 (ID: 1052419920)	Charger Configuration
<u>ChgCfgAbsorp</u>	0x11800 (ID: 2735)	Charger Configuration - Absorption
<u>ChgCfgBulk</u>	0x11700 (ID: 2734)	Charger Configuration - Bulk
<u>ChgCfgCvCi</u>	0x11C00 (ID: 2739)	Charger Configuration - Constant Voltage Constant Current
<u>ChgCfgEqualize</u>	0x11B00 (ID: 2738)	Charger Configuration - Equalize
<u>ChgCfgFloat</u>	0x11A00 (ID: 2737)	Charger Configuration - Float
<u>ChgCfgMultipleNodes</u>	0x12700 (ID: 2750)	Charger Configuration - Multiple Nodes
<u>ChgCfgOvercharge</u>	0x11900 (ID: 2736)	Charger Configuration - Overcharge
<u>ChgCtrlCmd</u>	0x13500 (ID: 1923696083)	Charger Control Command
<u>ChgCtrlCmd2</u>	0x1C800 (ID: -1979487)	Charger Control Command(Deux)
<u>ChgStatsAc</u>	0x1F01A (ID: 208033297)	Charger Statistics - AC
<u>ChgStatsAc2</u>	0x1F0C0 (ID: 131250206)	Charger Statistics - AC (Deux)
<u>ChgStatsBatt2</u>	0x1F0BE (ID: 1830828066)	Charger Statistics - Battery (Deux)
<u>ChgStatsDc</u>	0x1F021 (ID: 244344574)	Charger Statistics - DC
<u>ChgStatsDc2</u>	0x1F0C1 (ID: 1141402231)	Charger Statistics - DC (Deux)
<u>ChgStatsExtTemp</u>	0x1F023 (ID: -1262345440)	Charger Statistics - External Temperature
<u>ChgStatsIntTemp</u>	0x1F022 (ID: -6788613)	Charger Statistics - Internal Temperature
<u>ChgSts</u>	0x1F00E (ID: 2702)	Charger Status
<u>ClearCmd</u>	0x12800 (ID: 2751)	Clear Command
<u>CommStats</u>	0x1F002 (ID: 1045862584)	Communication Statistics
<u>ComplexReq</u>	0x1DE00 (ID: 1137167582)	Complex Request
<u>CtrlCmd</u>	0x14000 (ID: 1045509179)	Control Command
<u>DateTimeCfg</u>	0x14800 (ID: 673883469)	Date and Time Configuration
<u>DateTimeSts</u>	0x1F809 (ID: 2711)	Date and Time Status

<u>DcSrcCfgBatt</u>	0x11D00 (ID: 2740)	DC Source Configuration - Battery
<u>DcSrcCfgOi</u>	0x10400 (ID: 2715)	DC Source Configuration - Over Current
<u>DcSrcCfgOv</u>	0x10200 (ID: 2713)	DC Source Configuration - Over Voltage
<u>DcSrcCfgPvArray</u>	0x15300 (ID: 1081900322)	DC Source Configuration - Photovoltaic Array
<u>DcSrcCfgPvArray2</u>	0x1C300 (ID: 1137084597)	DC Source Configuration - Photovoltaic Array (Deux)
<u>DcSrcCfgRipOvPeak</u>	0x10500 (ID: 2716)	DC Source Configuration - Over Ripple Voltage - Peak
<u>DcSrcCfgUi</u>	0x10300 (ID: 2714)	DC Source Configuration - Under Current
<u>DcSrcCfgUv</u>	0x10100 (ID: 2712)	DC Source Configuration - Under Voltage
<u>DcSrcSts</u>	0x1F001 (ID: 2689)	DC Source Status
<u>DcSrcSts2</u>	0x1F0C5 (ID: -1392990766)	DC Source Status (Deux)
<u>DebugFP</u>	0x13900 (ID: -735169084)	Debug Fast-Packet
<u>DevPeekAddrReq</u>	0x12E00 (ID: 2757)	Development - Peek at Address Request
<u>DevPeekAddrRsp</u>	0x12F00 (ID: 2758)	Development - Peek at Address Response
<u>DevPokeAddr</u>	0x13000 (ID: 2759)	Development - Poke at Address
<u>DiagCntctCmd</u>	0x14B00 (ID: 1127726192)	Diagnostic - Contact Command
<u>DiagCntctSts</u>	0x14C00 (ID: 1054747237)	Diagnostic - Contact Status
<u>DiagFanCmd</u>	0x14D00 (ID: -803684431)	Diagnostic - Fan Command
<u>DiagIndicCmd</u>	0x14A00 (ID: 1049740530)	Diagnostic - Indicator Command
<u>DiagMfgCfg</u>	0x15000 (ID: 1054930712)	Diagnostic - Manufacturing Configuration
<u>EnDisCfg</u>	0x13400 (ID: 2763)	Enable/Disable Configuration
<u>EnDisCfgTod</u>	0x1C100 (ID: 1148416880)	Enable/Disable Configuration - Time-of-Day
<u>ExtTempCfgOt</u>	0x11400 (ID: 2731)	External Temperature Configuration - Over Temp
<u>ExtTempCfgUt</u>	0x11300 (ID: 2730)	External Temperature Configuration - Under Temp
<u>ExtTempStats2</u>	0x1F0C7 (ID: 1137176309)	External Temperature Statistics (Deux)
<u>ExtTempSts</u>	0x1F00C (ID: 2700)	External Temperature Status
<u>FaultLogSts</u>	0x1F013 (ID: 2705)	Fault Log Status
<u>FaultLogSts2</u>	0x1F8C8 (ID: 734599079)	Fault Log Status (Deux)
<u>FileDnldBlk</u>	0x13100 (ID: 2760)	File Download Block
<u>FileDnldInf</u>	0x13200 (ID: 2761)	File Download Information
<u>FileDnldSts</u>	0x13B00 (ID: 1042504085)	File Download Status
<u>GoCfgYourselfCmd</u>	0x1C200 (ID: 266445036)	Go Configure Yourself Command
<u>GridTieCfg</u>	0x1C000 (ID: 108087070)	Grid Tie Configuration
<u>GwCfg</u>	0x15C00 (ID: -1139074202)	Gateway Configuration
<u>GwCfgAddrTransOthIntf</u>	0x15B00 (ID: -1833646823)	Gateway Configuration - Address Translation - Other Interface
<u>GwCfgAddrTransXbIntf</u>	0x15800 (ID: 1087861528)	Gateway Configuration - Address Translation - <u>XanBus</u> Interface
<u>GwCfgIdTrans</u>	0x15D00 (ID: 1125336769)	Gateway Configuration - Identifier Translation

<u>GwCfgMsgFiltersOthIntf</u>	0x15A00 (ID: 1087922296)	Gateway Configuration - Message Filters - Other Interface
<u>GwCfgMsgFiltersXbIntf</u>	0x15900 (ID: -273192945)	Gateway Configuration - Message Filters - <u>XanBus</u> Interface
<u>GwStats</u>	0x1F025 (ID: 960950022)	Gateway Statistics
<u>GwSts</u>	0x1F024 (ID: 1162267449)	Gateway Status
<u>HvDcBusCfgOv</u>	0x11200 (ID: 2729)	High Voltage DC Bus Configuration - Over Voltage
<u>HvDcBusCfgUv</u>	0x11100 (ID: 2728)	High Voltage DC Bus Configuration - Under Voltage
<u>HvDcBusSts</u>	0x1F00B (ID: 2699)	High Voltage DC Bus Status
<u>HwRevSts</u>	0x1F810 (ID: 2764)	Hardware Revision Status
<u>IntTempCfgOt</u>	0x11600 (ID: 2733)	Internal Temperature Configuration - Over Temp
<u>IntTempCfgUt</u>	0x11500 (ID: 2732)	Internal Temperature Configuration - Under Temp
<u>IntTempStats2</u>	0x1F0C8 (ID: 1058348818)	Internal Temperature Statistics (Deux)
<u>IntTempSts</u>	0x1F00D (ID: 2701)	Internal Temperature Status
<u>InvCfg</u>	0x12200 (ID: 2745)	Inverter Configuration
<u>InvCfgLoadSense</u>	0x12100 (ID: 2744)	Inverter Configuration - Load Sense
<u>InvCtrlCmd</u>	0x13700 (ID: -767764456)	Inverter Control Command
<u>InvStatsAc</u>	0x1F010 (ID: 2109802576)	Inverter Statistics - AC
<u>InvStatsAc2</u>	0x1F0C2 (ID: 2704)	Inverter Statistics - AC (Deux)
<u>InvStatsDc</u>	0x1F01E (ID: 1137166235)	Inverter Statistics - DC
<u>InvStatsDc2</u>	0x1F0C3 (ID: 1080675796)	Inverter Statistics - DC (Deux)
<u>InvStatsExtTemp</u>	0x1F020 (ID: 1504636065)	Inverter Statistics - External Temperature
<u>InvStatsIntTemp</u>	0x1F01F (ID: -1299863995)	Inverter Statistics - Internal Temperature
<u>InvSts</u>	0x1F019 (ID: 2707)	Inverter Status
<u>InvSts2</u>	0x1F0BD (ID: 1142888998)	Inverter Status (Deux)
<u>InvStsLoadShareHS</u>	0x1F01D (ID: 1122570268)	Inverter Status - Load Share (High Speed)
<u>ISOAck</u>	0xE800 (ID: 2677)	ISO Acknowledgment
<u>ISOAddrClaim</u>	0xEE00 (ID: 2685)	ISO Address Claim
<u>ISOCmdAddr</u>	0xFED8 (ID: 2686)	ISO Commanded Address
<u>ISOReq</u>	0xEA00 (ID: 2678)	ISO Request 59904 ? 0xEA00
<u>ISOTpConnManAbort</u>	0xEC00 (ID: 2680)	ISO Transport Protocol, Connection Management - Abort
<u>ISOTpConnManBAM</u>	0xEC00 (ID: 2681)	ISO Transport Protocol, Connection Management - BAM
<u>ISOTpConnManCTS</u>	0xEC00 (ID: 2683)	ISO Transport Protocol, Connection Management - CTS
<u>ISOTpConnManEOM</u>	0xEC00 (ID: 2682)	ISO Transport Protocol, Connection Management - EOM
<u>ISOTpConnManRTS</u>	0xEC00 (ID: 2684)	ISO Transport Protocol, Connection Management -

		RTS
<u>ISOTpXfer</u>	0xEB00 (ID: 2679)	ISO Transport Protocol, Data Transfer
<u>MpptSts</u>	0x1F0C9 (ID: 1158870517)	Maximum Power Point Tracker Status
<u>NAMECfg</u>	0x13A00 (ID: 1039482662)	<u>NAME</u> Configuration
<u>OpsCfgOv</u>	0x11000 (ID: 2727)	Offline Power Supply Configuration - Over Voltage
<u>OpsCfgUv</u>	0x10F00 (ID: 2726)	Offline Power Supply Configuration - Under Voltage
<u>OpsSts</u>	0x1F00A (ID: 2698)	Offline Power Supply Status
<u>PerXmitCfg</u>	0x12D00 (ID: 2756)	Periodic Transmit Configuration
<u>PgnListRX</u>	0x1EE00 (ID: 1053465942)	PGN List - Received PGN's Group Function
<u>PgnListTX</u>	0x1EE00 (ID: 981003295)	PGN List - Transmitted PGN's Group Function
<u>PidCfg</u>	0x1C900 (ID: 1164846545)	PID Control Loop Configuration
<u>PidSts</u>	0x1F0CA (ID: 1164905695)	PID Control Loop Status
<u>ProdInfoSts</u>	0x1F014 (ID: 2755)	Product Information Status
<u>ResetCmd</u>	0x12400 (ID: 2747)	Reset Command
<u>RxToCfg</u>	0x13300 (ID: 2762)	Receive Timeout Configuration
<u>SimpleCalibCfg</u>	0x13800 (ID: 1074206464)	Simple Calibration Configuration
<u>SpsCfg</u>	0x1C600 (ID: -2091803581)	Secondary Power Supply Configuration
<u>SpsCfgTrig</u>	0x1C700 (ID: 1769777608)	Secondary Power Supply Configuration - Triggers
<u>SpsCtrlCmd</u>	0x1C500 (ID: 1172799996)	Secondary Power Supply Control Command
<u>SpsSts</u>	0x1F0C6 (ID: 1128010965)	Secondary Power Supply Status
<u>StateEvtLogSts</u>	0x1F80F (ID: 2742)	State/Event Log Status
<u>StateEvtLogSts2</u>	0x1F8C9 (ID: 1154728872)	State/Event Log Status(Duex)
<u>StringCfg</u>	0x1DD00 (ID: -844204281)	String Configuration
<u>Sts</u>	0x1F00F (ID: 2703)	Status
<u>SwVerSts</u>	0x1F80E (ID: 2741)	Software Version Status
<u>SysIdCfg</u>	0x12300 (ID: 2039152441)	System Identifier Configuration
<u>SysIdSts</u>	0x1F012 (ID: -1342434554)	System Identifier Status
<u>UiCfg</u>	0x14100 (ID: 1045598168)	User Interface Configuration
<u>UiCfgAssocLbl</u>	0x14400 (ID: -400446513)	User Interface Configuration - Association Labels
<u>UiCfgFaultBehaviour</u>	0x15400 (ID: -1288738708)	User Interface Configuration - Faults Behaviour
<u>UiCfgFaultTxt</u>	0x14500 (ID: 923217252)	User Interface Configuration - Faults Text
<u>UiCfgNodeLbl</u>	0x14300 (ID: 1607621062)	User Interface Configuration - Node Labels
<u>UiCfgSplashImg</u>	0x14200 (ID: 1702773405)	User Interface Configuration - Splash Image
<u>UiCfgWarnBehaviour</u>	0x15500 (ID: 1082490871)	User Interface Configuration - Warnings Behaviour
<u>UiCfgWarnTxt</u>	0x14600 (ID: -2079665515)	User Interface Configuration - Warnings Text
<u>UiSts</u>	0x1F01C (ID: 72829488)	User Interface Status
<u>WarnLogSts</u>	0x1F80C (ID: -859660729)	Warning Log Status
<u>WarnLogSts2</u>	0x1F8C7 (ID: -611781634)	Warning Log Status (Deux)